

Standard di sviluppo

GSTRUMSVIL - Guida allo sviluppo in BC - © SISTEMI S.p.A.

- [Nomenclatura oggetti AGIS](#)
- [Strutturazione del codice](#)
- [Entità \(Classi e Tipi dato\)](#)
- [Interfaccia Utente](#)
- [Servizi e importazioni](#)
- [Elaborazioni](#)
- [Gestione filesystem](#)
- [Lettura abilitazione IDS](#)

Nomenclatura oggetti AGIS

Di seguito la nomenclatura da dare agli oggetti AGIS:

Oggetto	Nomeclatura	Esempio
Classe	<Sigla prodotto>_<Entita>	ES_ARTICOLO
Tipo dato	<Sigla prodotto>TD_<Entita>	ESTD_ARTICOLO
Oggetto query	<Sigla prodotto>OBJQRY_<Entita>_<Contesto>	ESOBJQRY_ARTICOLO_ELENCO
Contenitore di classi	<Sigla prodotto>_<Contesto>_CL	ES_ANAGRAFICA_CL
Contenitore di classi video base	<Sigla prodotto>_<Contesto>_UI_BASE_CL	ES_ANAGRAFICA_UI_BASE_CL
Contenitore di classi video desktop	<Sigla prodotto>_<Contesto>_UI_CL	ES_ANAGRAFICA_UI_CL
Contenitore di classi video web	<Sigla prodotto>_<Contesto>_UI_WEB_CL	ES_ANAGRAFICA_UI_WEB_CL
Contenitore di classi video mobile	<Sigla prodotto>_<Contesto>_UI_MOB_CL	ES_ANAGRAFICA_UI_MOB_CL
Contenitore di tipi dato	<Sigla prodotto>_<Contesto>_TD	ES_ANAGRAFICA_TD
DLL metodi di business logic	<Sigla prodotto>_<Contesto>_BL	ES_ANAGRAFICA_BL
DLL metodi di videata	<Sigla prodotto>_<Contesto>_UI	ES_ANAGRAFICA_UI
DLL metodi della classe video	<Sigla prodotto>_<Contesto>_UI	ES_ANAGRAFICA_UI
DLL di videata web e mobile	<Sigla prodotto>_<Contesto>_UI_WEB	ES_ANAGRAFICA_UI_WEB
DLL classe video base	<Sigla prodotto>_<Contesto>_UI_BASE	ES_ANAGRAFICA_UI_BASE
DLL metodi di definizione query	<Sigla prodotto>_<Contesto>_QRY	ES_ANAGRAFICA_QRY
WebService	<Sigla prodotto>_SRV_<Contesto>	ES_SRV_ANAGRAFICA

 Nello sviluppo di applicazioni Web centralizzate, che non avranno mai validità Desktop, per le classi video e i metodi di videata esisteranno solo le DLL con suffisso _UI e i contenitori di classi con suffisso _UI_CL. Tutte le altre declinazioni per web/mobile non esistono.

Strutturazione del codice

- [Regole generali](#)
- [Variabili](#)
- [Letture da database](#)

Regole generali

Sui nuovi sorgenti va usato l'attributo STRICT su `'@PG`, `'@PGVID` e `'@PGQUERY`, che guida all'uso degli ultimi standard di sviluppo.

Tramite esso si è obbligati ad utilizzare le `'@SRP` e quindi le variabili locali.

Le strutture dinamiche e le specifiche FINDDBDATA e GETDBDATA non vanno più usate.

Al loro posto va utilizzata la programmazione ad oggetti e le query per la lettura del database slegata dalle entità.

Indentazione del codice

Le righe di inizio/fine sorgente (`'@PG` / `'@PGVID` / `'@PGQUERY` e End) e inizio/fine subroutine (`'@SRP` e Return) devono cominciare da colonna 1.

Le righe al loro interno devono avere un'indentazione e quindi cominciare da colonna 4.

Variabili

Nomenclatura

Per quanto concerne la nomenclatura delle variabili, tutto ciò che ha un utilizzo locale va denominato con la prima lettera minuscola (se il nome è composto da più parole, l'iniziale delle successive deve essere maiuscola). Invece, tutto ciò che ha un utilizzo condiviso e valido per tutto il sorgente, deve cominciare con la prima lettera maiuscola.

Variabili locali

Con l'avvento delle `'@SRP` le variabili sono diventate locali e utilizzabili soltanto nel contesto in cui sono dichiarate.

Per convenzione, il nome di una variabile locale dovrebbe iniziare sempre con la prima lettera minuscola.

Se il nome è composto da più parole, l'iniziale delle successive deve essere maiuscola.

Variabili Shared

Per convenzione, il nome di una variabile Shared dovrebbe iniziare sempre con la prima lettera maiuscola, in modo tale da differenziarsi dalle variabili locali (che iniziano con la lettera minuscola).

Parametri della sub (`'@SBP`)

Il nome dei parametri della sub, che sono sempre utilizzabili in tutto il sorgente, dovrebbe iniziare sempre con la prima lettera maiuscola.

Parametri delle subroutine interne (`'@SRP`)

Il nome dei parametri delle subroutine interne, che sono di fatto variabili locali, dovrebbe iniziare sempre con la prima lettera minuscola.

Dichiarazione e utilizzo

Per la dichiarazione di variabili semplici i tipi da utilizzare sono cinque:

- INT
- STRING
- DOUBLE
- PERIOD
- DATE

Per la dichiarazione di stringhe va sempre usato STRING, non più BLOB o il tipo BCSTRING.

Non vanno più usati neanche gli altri tipi dato di ambiente BCINT, BCDOUBLE, BCDATE e BCPERIOD, se non quando è necessario dichiarare una collezione.

Il booleano non è un tipo nativo quindi per dichiarare una variabile booleana va usato il tipo dato BCBOOL.

Variabili Shared

L'uso dell'attributo SHARED nella dichiarazione di una variabile serve quando si vuole rendere la variabile condivisa e utilizzabile in tutto il sorgente.

E' indicato quando se ne deve fare un largo utilizzo in molteplici '@SRP del sorgente.

Per pulizia le variabili SHARED devono essere per forza dichiarate subito sotto la '@SBP, altrimenti verrà emesso un errore in traduzione.

Lecture da database

Per la lettura di record da database, nel caso non si voglia leggere un intero oggetto o collezione, vanno usate le query e il DATAREADER con i suoi metodi.

Caso con lettura di più di una riga:

BC

```
DIM qryLettura[DATAREADER[BLDOBJQRY_DATI_LEGGERE()]]

While qryLettura.Next()

    ' Lettura dati
    If qryLettura.Campol > 0 Then

        EndIf

    EndWhile
```

Caso con lettura di più della prima riga

BC

```
DIM qryLettura[DATAREADER[BLDOBJQRY_DATI_LEGGERE()]]

qryLettura.First()
```

Entità (Classi e Tipi dato)

- [Classe](#)
- [Tipo dato](#)

- [Classi di filtri](#)
- [Classi Helper](#)
- [Enumerato Errori](#)
- [Logica di definizione di un metodo](#)

Classe

- [Metodi](#)
- [Attributi delle proprietà](#)

Metodi

Metodo	Contesto	Tipo	Validità	Descrizione
Carica	Oggetto	BL	Multiplatforma	Caricamento dati dell'entità
Elimina	Oggetto	BL	Multiplatforma	Eliminazione dati dell'entità
EsisteSuDb	Statico	BL	Multiplatforma	Controlla esistenza entità su Db
Salva	Oggetto	BL	Multiplatforma	Salvataggio dati dell'entità
AttivaCache	Statico	BL	Multiplatforma	Attivazione cache dell'entità
AttivaCacheUtilizzate	Statico	BL	Multiplatforma	Attivazione cache utilizzate dall'entità
Deriva	Oggetto	BL	Multiplatforma	Derivazione dati dell'entità
Dettaglio	Oggetto	UI	Desktop	Videata di dettaglio dell'entità
DettaglioWeb	Oggetto	UI	Web	Videata di dettaglio dell'entità in versione Web
DettaglioMobile	Oggetto	UI	Mobile	Videata di dettaglio dell'entità in versione Mobile
Duplica	Oggetto	BL	Multiplatforma	Duplicazione dell'entità
Elenco	Statico	UI	Desktop	Videata di elenco dell'entità, usata anche ai fini della ricerca
ElencoWeb	Statico	UI	Web	Videata di elenco dell'entità in versione Web
ElencoMobile	Statico	UI	Mobile	Videata di elenco dell'entità in versione Mobile
Gestione	Statico	BL	Desktop	Videata di gestione dell'entità (metodo wrapper che richiama Elenco)
Inizializza	Oggetto	BL	Multiplatforma	Inizializzazione dell'entità
PulisciCache	Statico	BL	Multiplatforma	Pulizia cache dell'entità
PulisciCacheUtilizzate	Statico	BL	Multiplatforma	Pulizia cache utilizzate dall'entità
QueryElenco	Statico	BL	Multiplatforma	Definizione della query usata dall'Elenco
QueryElencoAggiornamento	Statico	BL	Multiplatforma	Ridefinisce la query dell'Elenco per effettuare l'aggiornamento
Valida	Oggetto	BL	Multiplatforma	Validazione dell'entità
ImpostaAttributoObbligatorio	Oggetto	BL	Multiplatforma	Impostazione attributo #REQUIRED sulle proprietà dell'oggetto
ImpostaAttributiVideo	Oggetto	BL	Multiplatforma	Impostazione attributi video sulle proprietà dell'oggetto
ImpostaAttributiVideoDaProprieta	Oggetto	BL	Multiplatforma	Wrapper che richiama ImpostaAttributiVideo a partire da altra proprietà
IsProprietaGestita	Oggetto	BL	Multiplatforma	Determina se una proprietà è gestita
IsProprietaProtetta	Oggetto	BL	Multiplatforma	Determina se una proprietà è protetta
Blocca	Statico	BL	Multiplatforma	Blocca la variazione dell'entità
Sblocca	Statico	BL	Multiplatforma	Sblocca la variazione dell'entità
Metodi di controllo	Oggetto	BL	Multiplatforma	

 I metodi Carica, Elimina e Salva solitamente non vanno creati su classi che rappresentano sottoentità. La loro creazione va fatta soltanto quando è necessario sostituire i metodi base integrandoli con della logica applicativa.

Carica

E' il metodo che si occupa di caricare l'entità dal database.
Salvo casi particolari, deve richiamare il metodo di ambiente `LoadFromDb()`.
Il metodo Carica andrebbe creato solo su classi che rappresentano entità autonome e non sottoentità.
Per le sottoentità la creazione va fatta soltanto quando è necessario sostituire il metodo base integrandolo con della logica applicativa (se è sufficiente leggere dei campi in più in query va usata la query di caricamento).

 In caso di mancato caricamento dati non vanno settati errori sull'oggetto. Il valore di ritorno del metodo da questa informazione e il chiamante lo gestisce.



Nel caso di classe mappata su descrittore che prevede una chiave tecnica e una applicativa il metodo Carica opera sulla chiave primaria, ovvero quella tecnica. Può però nascere l'esigenza di dover caricare i dati a partire dalla chiave applicativa. In questi casi va creato uno specifico metodo "CaricaDa<xxxx>" che al suo interno esegue una lettura del primo elemento (metodo First() del DATAREADER) per determinare il valore della chiave primaria e successivamente richiama il metodo Carica.

Nel caso in cui sia necessario permettere di caricare la classe usando un gruppo diverso da quello corrente, è possibile aggiungere il gruppo nella firma del metodo, versionato. Il gruppo va poi impostato sul campo GR dell'oggetto prima di chiamare il LoadFromDb().

Firma

Definizione BC				
@SBP	This[CLASSE[<classe>]]		[INOUT]	_
	Chiave[INT]		[IN]	_
	Livello[INT]		[IN]	_
	UtilizzaCache[TIPO[BCBOOL]]		[IN]	_
	Errori[Classe[BC_VID_ERROR] COLLEZIONE]		[OUT]	_
	AnnullatoLogicamente[TIPO[BCBOOL]]	[VER=1]	[IN]	
AS Caricato[TIPO[BCBOOL]]				

Parametri

This

Oggetto da caricare.

Chiave

Uno o più parametri che rappresentano i campi chiave.

Livello

Indica il livello di lettura dell'entità.

UtilizzaCache

Indica se utilizzare la cache (se attiva) o forzare la lettura da DB.

Errori

Collezione di errori da popolare nel caso in cui si riscontrino anomalie nel caricamento dell'oggetto.

[AnnullatoLogicamente]

Flag che Indica se caricare anche le entità logicamente annullate. E' un parametro opzionale da indicare soltanto se sull'entità le eliminazioni non vanno a cancellare il record su DB ma solo ad annullarlo.

Caricato

Valore di ritorno che indica se l'oggetto è stato caricato con successo o meno.

Elimina

E' il metodo che si occupa di eliminare l'entità dal database.

Salvo casi particolari, deve richiamare il metodo di ambiente DelFromDb().

Il metodo Elimina andrebbe creato solo su classi che rappresentano entità autonome e non sottoentità.

Per le sottoentità la creazione va fatta soltanto quando è necessario sostituire il metodo base integrandolo con della logica applicativa.

Se prima di eliminare i dati dal database è necessario eseguire controlli referenziali essi vanno eseguiti qui dentro e, in caso di blocco dell'operazione di eliminazione, deve essere popolata la collezione di errori prevista in firma del metodo. (ad esempio non si può eliminare un cliente se sono già stati caricati documenti intestati ad esso). Sarà cura del chiamante controllare la presenza di eventuali errori.

Da prestare attenzione alle prestazioni valutando la creazione di indici sulle foreign key su cui viene effettuato il controllo.

Firma

Definizione BC				
'@SBP	This[CLASSE[<classe>]]	[INOUT]	_	
	UtilizzaTransazione[TIPO[BCBOOL]]	[IN]	_	
	Errori[Classe[BC_VID_ERROR] COLLEZIONE]	[OUT]	_	
	IgnoraWarning[TIPO[BCBOOL]]	[IN]	[VER=1]	[DEFAULT=#True] _
	AS Eliminato [TIPO[BCBOOL]]			

Parametri

This

Oggetto da eliminare. L'oggetto dovrà avere almeno i campi chiave valorizzati.

UtilizzaTransazione

Indica se aprire una transazione per l'eliminazione dell'oggetto.

Errori

Collezione di errori da popolare nel caso in cui si riscontrino anomalie nell'eliminazione dell'oggetto.

IgnoraWarning

Parametro opzionale che indica se ignorare eventuali warning riscontrati in eliminazione. Di default è True quindi i warning vengono ignorati. Se il chiamante vuole gestirli deve passare questo parametro a False e poi emettere i warning. Se essi vengono forzati dall'utente deve essere richiamato il metodo Elimina con True.

Eliminato

Valore di ritorno che indica se l'oggetto è stato eliminato con successo o meno.

Esempio

BC				
'@SBP	This[CLASSE[]]	[INOUT]	_	
	UtilizzaTransazione[TIPO[BCBOOL]]	[IN]	_	
	Errori[CLASSE[BC_VID_ERROR] COLLEZIONE]	[OUT]	_	
	IgnoraWarning[TIPO[BCBOOL]]	[IN]	[VER=1] [DEFAULT=#True]	_
	AS Eliminato[TIPO[BCBOOL]]			
<pre>Eliminato = This.DelFromDb(UtilizzaTransazione)</pre>				
	End			

EsisteSuDb

E' il metodo che controlla l'esistenza dell'entità su Db.

Al suo interno richiama il metodo di ambiente ExistsOnDb().

Va fatto solo sulle classi pubbliche perché l'ExistsOnDb ha visibilità interna, quindi serve a permettere l'uso a chi importa la classe.

E' un metodo statico.

Firma

Definizione BC			
'@SBP	Chiave[INT]	[IN]	_
	AS Esiste[TIPO[BCBOOL]]		

Parametri

Chiave

Uno o più parametri che rappresentano i campi chiave.

Esiste

Valore booleano che restituisce se l'entità è stata trovata su Db.

Salva

E' il metodo che si occupa di salvare l'entità sul database.

Salvo casi particolari, deve richiamare il metodo di ambiente [SaveToDb\(\)](#).

Il metodo Salva andrebbe creato solo su classi che rappresentano entità autonome e non sottoentità.

Per le sottoentità la creazione va fatta soltanto quando è necessario sostituire il metodo base integrandolo con della logica applicativa.

Firma

Definizione BC

'@SBP	This[CLASSE[<classe>]]	[INOUT]	_
	UtilizzaTransazione[TIPO[BCBOOL]]	[IN]	

Parametri

This

Oggetto da eliminare. L'oggetto dovrà avere almeno i campi chiave valorizzati.

UtilizzaTransazione

Indica se aprire una transazione per l'eliminazione dell'oggetto.

Esempio

BC

```
'@SBP This[CLASSE[]] [INOUT] _
    UtilizzaTransazione[TIPO[BCBOOL]] [IN]

    This.SaveToDb(UtilizzaTransazione)

End
```

Blocca

Metodo statico che imposta un lock sul database per bloccare la variazione dell'entità da altri operatori/programmi.

Il lock acquisito dovrà poi essere sbloccato tramite il metodo [Sblocca](#).

Firma

Definizione BC

'@SBP	Chiave	[IN]	_
	Errore[Classe[BC_VID_ERROR] COLLEZIONE]	[OUT]	_
	Lock[CLASSE[BC_DA_APPLOCK]]	[OUT]	_
	AS BloccoAcquisito[TIPO[BCBOOL]]		

Parametri

Chiave

Uno o più parametri che rappresentano i campi chiave.

Errore

Se il blocco non viene acquisito, l'oggetto errore viene valorizzato con l'errore rilevato (entità in uso oppure transazione in corso). Al messaggio di errore deve essere aggiunto il prefisso relativo all'entità che sto trattando (es. Documento xxx)

Lock

Oggetto in output che contiene le informazioni del lock. Viene valorizzato sia in caso di lock acquisito che in caso di lock occupato.

In caso di lock acquisito l'oggetto dovrà poi essere passato al metodo Sblocca per sbloccare l'entità.

BloccoAcquisito

Indica se il lock è stato acquisito

```
BC

'@PG BLD_FATTURA_BLOCCA STRICT[3]      ' Blocco entita con lock applicativo

'@SBP IdDoc[INT]                        [IN]      _ ' Chiave
      Errore[CLASSE[BC_VID_ERROR]]      [OUT]     _ ' Eventuale errore riscontrato
      Lock[CLASSE[BC_DA_APPLOCK]]       [OUT]     _ ' Oggetto del lock acquisito
      AS BloccoAcquisito[TIPO[BCBOOL]]

DIM nomeLock[STRING] = BCDatabaseInfo.GetName()
if BcContextInfo.IsMultiIstanza() Then nomeLock += "_" + BcContextInfo.GetDominioMultiIstanza().ToString()
nomeLock += "_BLD_FATTURA_" + IdDoc.ToString()
' Se l'entità è di gruppo va aggiunto anche il gruppo
nomeLock += "_" + BcContextInfo.GetGruppo()

BloccoAcquisito = Lock.GetAppLock(nomeLock, Errore [OUT])

If Not BloccoAcquisito Then Errore.Msg = "Documento " + IdDoc.ToString() + " " + Errore.Msg

End
```

Deriva

E' il metodo che si occupa di derivare dati a partire da una specifica proprietà.

Viene richiamato a seguito della validazione se non ci sono errori.

Quando viene assegnato il valore di una proprietà, su quest'ultima va richiamato il metodo Valida() e il metodo Deriva (soltanto se dopo il Valida non sono stati riscontrati errori). Di fatto per ogni proprietà contiene la logica di assegnazione dei dati completa.

Firma

Definizione BC			
'@SBP	This[CLASSE[<classe>]]	[INOUT]	_
	ProprietaCorrente[STRING]	[IN]	_
	ProprietaDerivate[TIPO[BCSTRING] COLLEZIONE] [OUT]		

Parametri

This

Oggetto sul quale effettuare la derivazione dei dati.

ProprietaCorrente

Id della proprietà corrente per la quale derivare altri dati. Va confrontato con il PropertyId di ogni proprietà. Se blank, vanno effettuate tutte le derivazioni.

ProprietaDerivate

Elenco di proprieta derivate. Questo elenco serve a sapere quali proprietà sono state derivate e viene utilizzato sia nei Deriva delle classi specializzate (classe video o classe derivata) che nel metodo ImpostaAttributiVideoDaProprieta

Esempio

BC

```
'@SBP This[CLASSE[]] [INOUT] _
  ProprietaCorrente[STRING] [IN] _ ' Proprietà per cui eseguire la
derivazione dei dati
  ProprietaDerivate[TIPO[BCSTRING] COLLEZIONE] [OUT]

  ProprietaDerivate.Add(ProprietaCorrente)

Switch ProprietaCorrente

  ' Inserire qui il richiamo alle SRP per la derivazione dei dati delle singole proprietà

Case This.PROP1.PropertyId() DerivaDatiProp1()

EndSwitch

'@SRP DerivaDatiProp1()

  ' Derivo la proprietà collegata solo se devo assegnarle un valore differente
If This.CAMPO2 <> VALORE Then

  This.CAMPO2 = VALORE
  ValidaProprietaDerivata(This.CAMPO2.PropertyId())

EndIf

Return

'@SRP ValidaProprietaDerivata(idProprieta[STRING] [IN])

This.Valida(idProprieta)
If Not This.HasError(idProprieta, #ERRORE) Then

  This.Deriva(idProprieta, proprietaDerivateDeriva[TIPO[BCSTRING] COLLEZIONE] [OUT])

  proprietaDerivateDeriva.ForEach(idProprieta, If Not ProprietaDerivate.Exists(idProprieta) Then
ProprietaDerivate.Add(idProprieta))

EndIf

Return
```

Riferimenti

Metodo Valida

Estendere la derivazione di proprietà di una sottoclasse

In determinati contesti può essere necessario estendere le logiche di derivazione di alcune proprietà di una sottoclasse.

Un caso di esempio è la classe video, per la quale può essere necessario estendere la derivazione di una proprietà del modello dati in funzione di logiche specifiche della videata.

Nel fare questa operazione bisogna tenere conto di alcuni accorgimenti necessari a far funzionare correttamente il giro di derivazione dati.

Nell'esempio viene simulato di dover estendere, sulla classe video, la derivazione di alcune proprietà del modello dati.

Derivazione sottoclasse specifica (in questo caso il modello dati):

BC

```
' Richiamo la derivazione sulla sottoclasse
This.ModelloDati.Deriva(ProprietaCorrente, proprietaDerivate[TIPO[BCSTRING] COLLEZIONE] [OUT])

' Richiamo la derivazione aggiuntiva delle proprietà se essa è stata derivata
If proprietaDerivate.Exists(This.ModelloDati.DurataPausa.PropertyId()) Then Deriva_ModelloDati_DurataPausa()
If proprietaDerivate.Exists(This.ModelloDati.DurataPrestazione.PropertyId()) Then Deriva_ModelloDati_DurataPrestazione()
```

L'impostazione delle proprietà derivate viene fatta nel metodo Deriva.

Riferimenti

Estendere la validazione di proprietà di una sottoclasse

Dettaglio

Videata di dettaglio dell'entità.

E' il metodo che si occupa di esporre a video l'entità. L'oggetto deve arrivare al metodo già caricato.

In caso di utilizzo del modello video, esso deve essere definito e popolato nella SRP Load.

Nel caso di dettaglio di sottoentità, nella Load va attivato lo snapshot tramite InitializeSnapshot().

In uscita poi si può sfruttare il metodo IsModifiedSnapshot per sapere se sono state fatte modifiche e il metodo RestoreSnapshot() per annullarle.

Per le entità non è necessario creare lo snapshot, è sufficiente usare i metodi IsModified() e Restore().

Restituisce un valore che indica l'azione compiuta.

Firma

Definizione BC				
'@SBP	This[CLASSE[<classe>]]	[INOUT]		-
	Modalita[TIPO[BCTD_MODALITADETTAGLIO]]	[IN]		-
	VariazioneAttiva[TIPO[BCBOOL]]	[IN]	[VER=1]	-
	AS Azione[TIPO[BCTD_AZIONEDETTAGLIO]]			

Parametri

This

Oggetto da esporre a video. L'oggetto deve arrivare già caricato.

Modalita

Indica la modalità della videata di dettaglio (Caricamento, Variazione, SoloVisualizzazione).

Va usato per condizionare la visibilità dei bottoni in ribbon/toolbar e l'output della videata.

VariazioneAttiva

Flag da utilizzare in modalità Variazione, per indicare se mostrare la videata con la variazione già attiva oppure da attivare tramite tasto Varia.

Il parametro è opzionale e va previsto in firma solo se applicativamente si decide di offrire questa scelta al chiamante.

Azione

Valore di ritorno che indica l'azione compiuta sul dettaglio (Nessuna, Dati confermati, Dati eliminati).

Duplica

Metodo di duplicazione dell'entità.

Si occupa di completare il caricamento dell'oggetto di partenza e poi copiarlo su un nuovo oggetto cambiando la chiave e impostando lo stato come nuovo.

Il nuovo oggetto duplicato viene restituito come valore di ritorno.

Per completare il caricamento dell'oggetto di partenza va chiamato il metodo `Carica()` con livello -1. Se presenti estensioni personalizzate (`_PERS`) saranno caricate contestualmente ai dati letti dalle tabelle standard.

Firma

Definizione BC			
@SBP	This[CLASSE[<classe>]]	[INOUT]	_
	NuovaChiave	[IN]	_
AS Duplicato[CLASSE[<classe>]]			

Parametri

This

Oggetto da duplicare.

NuovaChiave

Uno o più parametri che rappresentano i campi chiave della nuova entità.

Duplicato

Oggetto duplicato.

Esempio

BC
<pre>'@SBP This[CLASSE[]] [INOUT] _ NuovaChiave [IN] _ AS Duplicato[CLASSE[]] ' Completo il caricamento della classe caricando anche eventuali personalizzazioni This.Carica(This.Chiave, -1, DisabilitaCache, Errori, NoClear) ' Duplico l'entità, aggiorno la chiave e imposto lo stato nuovo Duplicato = This Duplicato.Chiave = NuovaChiave Duplicato.UpdateKey() Duplicato.Setstatenew()</pre> End

Elenco

Il metodo Elenco va fatto sulla classe e deve essere un metodo **statico** di videata, nella quale viene esposto l'elenco delle entità.

Il metodo gestisce sia la ricerca che la gestione dell'entità, centralizzando la videata.

E' un metodo privato e va richiamato soltanto dai suoi metodi "wrapper" `Gestione()`, `Ricerca()`, `RicercaMultisel()`.

Firma

Definizione BC				
'@SBP	Modalita[TIPO[BCTD_MODALITAELENCO]]	[IN]	—	
	MultiSel[TIPO[BCBOOL]]	[IN]	—	
	CodPosIniziale[TIPO[<tipodato>]]	[IN]	—	
	CodPreSelezionati[TIPO[BCSTRING]]	[IN]	—	
	CodSelezionati[TIPO[BCSTRING]]	[OUT]	—	
	Filtri	[INOUT]		

Parametri

Modalita

Indica la modalità di chiamata. E' disponibile il tipo dato BCTD_MODALITAELENCO che prevede i valori Ricerca, RicercaConGestione, Gestione, Interrogazione.

Serve a pilotare alcuni aspetti grafici ovvero la visibilità dei bottoni in Ribbon e il titolo della videata e/o dei pannelli.

Multisel

Indica se gestire la multi selezione. Non è necessario se non si intende gestirla.

CodPosIniziale

Codice per il posizionamento iniziale in griglia.

CodPreSelezionati

Stringa di ingresso contenente l'elenco dei codici (separati da una virgola) selezionati ad ingresso videata.

Se la griglia è caricata da query, questa stringa va usata per impostare un campo calcolato in query con alias ZSEL, che servirà a selezionare le righe in griglia.

Non è necessario se non si gestisce la multi selezione.

CodSelezionati

Stringa di uscita che conterrà l'elenco dei valori selezionati in griglia (separati da una virgola).

Filtri

Elenco di filtri da applicare.

Può essere una classe applicativa di filtri o un elenco di parametri semplici.

Riferimenti

Metodo Gestione

Metodo Ricerca

Metodo RicercaMultisel

Gestione

Il metodo Gestione deve essere un metodo statico della classe.

Si tratta di un metodo "wrapper" che richiama il metodo statico Elenco() nel quale è centralizzata la videata.

Firma

Definizione BC				
'@SBP	Filtri	[INOUT]	[VER=1]	—
	SolaInterrogazione[TIPO[BCBOOL]]	[IN]	[VER=2]	

Parametri

Filtri

Elenco di filtri da applicare.

Può essere una classe applicativa di filtri o un elenco di parametri semplici.

SolaInterrogazione

Indica se entrare in gestione in modalità Interrogazione, quindi senza la possibilità di variare o eliminare.

Esempio

BC

```
'@SBP Filtri [INOUT] [VER=1] _
  SolaInterrogazione[TIPO[BCBOOL]] [IN] [VER=2]

  If SolaInterrogazione Then

    <Classe>.Elenco(BCTD_MODALITAELENCO.Interrogazione, #True, 0, "", ? [OUT], Filtri [INOUT])

  Else

    <Classe>.Elenco(BCTD_MODALITAELENCO.Gestione, #True, 0, "", ? [OUT], Filtri [INOUT])

  Endif

End
```

Riferimenti

Metodo [Elenco](#)

ImpostaAttributoObbligatorio

E' il metodo che si occupa di impostare i campi obbligatori **condizionali**.
Il suo scopo è quello di **centralizzare** la logica sia per l'interfaccia utente che per la logica applicativa.
In ambito UI l'attributo #REQUIRED viene usato per evidenziare i campi a video (metodo [ImpostaAttributiVideo\(\)](#)).
In ambito BL viene usato da metodo ValidateFromClass, richiamato nel [Valida\(\)](#), per impostare l'errore se il campo non è valorizzato.

Se una proprietà è sempre obbligatoria non va gestita in questo metodo ma deve essere impostata come obbligatoria nella definizione della classe.

In caso di oggetto composto, il metodo deve impostare l'attributo obbligatorio solo sulle proprietà semplici del primo livello e non deve scendere sui sottolivelli.

Firma

Definizione BC				
'@SBP	This[CLASSE[<classe>]]	[INOUT]	_	
ProprietaCorrente[STRING] [IN]				

Parametri

This
Oggetto sul quale impostare l'attributo #REQUIRED.

ProprietaCorrente
Id della proprietà corrente per la quale valutare l'impostazione dell'attributo #REQUIRED. Va confrontato con il PropertyId di ogni proprietà. Se blank, richiama l'impostazione degli attributi di tutte le proprietà.

Riferimenti

Metodo [ImpostaAttributiVideo\(\)](#)
Metodo [Valida\(\)](#)

ImpostaAttributiVideo

E' il metodo che si occupa di impostare gli attributi video sulle proprietà dell'oggetto. Il metodo può essere richiamato per una specifica proprietà o per l'intero oggetto.

Se sull'entità è presente un modello video, il metodo va creato sul modello video.

Solo per i prodotti personalizzabili, che non prevedono la classe VID_BASE, il metodo va creato anche sul modello dati per condividere impostazioni comuni tra gli eventuali modelli video desktop/web/mobile.

Il metodo deve essere richiamato nella subroutine interna associata al parametro ONLOAD della @DEFVID.

Il richiamo per la singola proprietà invece avviene sempre e solo dentro il metodo `ImpostaAttributiVideoDaProprieta()`, che a sua volta è chiamato nel sorgente della videata, all'interno del parametro `IMPOSTAATTRIBUTIVIDEODAPROPRIETA[]` della specifica @DEFVID

Per impostare gli attributi #HIDDEN, #DISABLED e #OUTPUT, vanno richiamati i metodi `IsProprietaGestita()` e `IsProprietaProtetta()`.

Per impostare l'attributo #REQUIRED, va richiamato il metodo `ImpostaAttributoObbligatorio()`.

Il metodo deve impostare gli attributi video soltanto per le proprietà semplici che vengono esposte a video e non deve scendere sui figli.

Firma

Definizione BC			
'@SBP	This[CLASSE[<classe>]]	[INOUT]	_
	ProprietaCorrente[STRING]	[IN]	

Parametri

This

Oggetto sul quale impostare gli attributi.

ProprietaCorrente

Id della proprietà corrente per la quale valutare l'impostazione degli attributi. Va confrontato con il PropertyId di ogni proprietà. Se blank, richiama l'impostazione degli attributi di tutte le proprietà.

Esempio

BC

```
'@SBP This[CLASSE[]] [INOUT] _
    ProprietaCorrente[STRING] [IN] ' Proprieta corrente

If ProprietaCorrente = "" Then

    ImpostaSuModelloDati(This.ModelloDati.PROP1.PropertyId())
    Imposta(This.PROP1.PropertyId())
    Imposta(This.PROP2.PropertyId())

Else

    Switch ProprietaCorrente
        Case This.ModelloDati.PROP1.PropertyId() GoSub ImpostaSuModelloDati(ProprietaCorrente)
        Case This.PROP1.PropertyId() GoSub Imposta(ProprietaCorrente)
        Case This.PROP2.PropertyId() GoSub Imposta(ProprietaCorrente)
    EndSwitch

EndIf
This.ImpostaAttributoObbligatorio(ProprietaCorrente)

End

'@SRP ImpostaSuModelloDati(propertyId[STRING] [IN])

If This.ModelloDati.IsProprietaGestita(propertyId, ? [OUT], IsDisabilitata[TIPO[BCBOOL]]
[OUT]) Then
    This.ModelloDati.SetAttributeBool(propertyId, #HIDDEN, #False)
    This.ModelloDati.SetAttributeBool(propertyId,
#OUTPUT, This.ModelloDati.IsProprietaProtetta(propertyId))
Else
    This.ModelloDati.SetAttributeBool(propertyId, #HIDDEN, Not IsDisabilitata)
    This.ModelloDati.SetAttributeBool(propertyId, #OUTPUT, #False)
EndIf
This.ModelloDati.SetAttributeBool(propertyId, #DISABLED, IsDisabilitata)

Return

'@SRP Imposta(propertyId[STRING] [IN])

If This.IsProprietaGestita(propertyId, ? [OUT], IsDisabilitata[TIPO[BCBOOL]] [OUT]) Then
    This.SetAttributeBool(propertyId, #HIDDEN, #False)
    This.SetAttributeBool(propertyId, #OUTPUT, This.IsProprietaProtetta(propertyId))
Else
    This.SetAttributeBool(propertyId, #HIDDEN, Not IsDisabilitata)
    This.SetAttributeBool(propertyId, #OUTPUT, #False)
EndIf
This.SetAttributeBool(propertyId, #DISABLED, IsDisabilitata)

Return
```

Riferimenti

- Metodo [Deriva](#)
- Metodo [ImpostaAttributiVideoDaProprieta](#)
- Metodi [IsProprietaGestita](#) e [IsProprietaProtetta](#)
- Metodo [ImpostaAttributoObbligatorio](#)
- [Impostazione stati video dei controlli](#)
- [Attributi delle proprietà](#)

ImpostaAttributiVideoDaProprieta

Metodo wrapper che si occupa di richiamare il metodo [ImpostaAttributiVideo\(\)](#) in seguito al cambio di valore di una o più proprietà.

Se sull'entità è presente un modello video, il metodo va creato solo sul modello video.

Il metodo va richiamato nel sorgente della videata, all'interno del parametro IMPOSTAATTRIBUTIVIDEODAPROPRIETA[] della specifica @DEFVID

Firma

Definizione BC			
'@SBP	This[CLASSE[<classe>]]	[INOUT]	_
	ProprietaDerivate[TIPO[BCSTRING] COLLEZIONE]	[INOUT]	

Parametri

This

Oggetto sul quale impostare gli attributi.

ProprietaDerivate

Elenco di proprietà che sono state derivate. Su queste proprietà va chiamato l'ImpostaAttributiVideo per le proprietà collegate.

Esempio

BC
<pre>'@SBP This[CLASSE[]] [INOUT] _ ProprietaDerivate[TIPO[BCSTRING] COLLEZIONE] [INOUT] SHARED DIM ProprietaElaborate[TIPO[BCSTRING] COLLEZIONE] If ProprietaDerivate.Exists(This.PROP1.PropertyId()) Then ' Richiamo l'impostazione degli attributi video sulle proprietà collegate ImpostaAttributi(This.PROP2.PropertyId()) Endif End '@SRP ImpostaAttributi(idProprieta[STRING] [IN]) If Not ProprietaElaborate.Exists(idProprieta) Then ProprietaElaborate.Add(idProprieta) This.ImpostaAttributiVideo(idProprieta) EndIf Return</pre>

Riferimenti

Metodo Deriva

Metodo ImpostaAttributiVideo

IsProprietaGestita

Contiene la **logica applicativa** che determina se una proprietà è gestita o meno.

Il suo scopo è quello di **centralizzare** tale logica sia per uso UI che BL:

In ambito UI viene richiamato dal metodo ImpostaAttributiVideo() per sapere se impostare l'attributo #HIDDEN.

In ambito BL viene richiamato per sapere se è una proprietà è gestita in quel contesto.

Per centralizzazione delle condizioni e ottimizzazione del programma, è possibile aggiungere il parametro IsDisabilitata come parametro di output versionato.

Tale parametro va utilizzato soltanto dalla parte video nel metodo ImpostaAttributiVideo().

Firma

Definizione BC				
'@SBP	This[CLASSE[<classe>]]	[INOUT]		_
	ProprietaCorrente[STRING]	[IN]		_
	Errore[CLASSE[BC_VID_ERROR]]	[OUT]		_
	IsDisabilitata[TIPO[BCBOOL]]	[OUT]	[VER=1]	_
	AS Gestita[TIPO[BCBOOL]]			

Parametri

This

Oggetto sul quale impostare gli attributi.

ProprietaCorrente

Id della proprietà corrente per la quale valutare se è gestita. Va confrontato con il PropertyId di ogni proprietà che può avere una condizione di non gestita.

Errore

Eventuale errore che indica il motivo per cui la proprietà non è gestita.
L'errore viene emesso da questo metodo perché deve essere il più possibile parlante.
L'errore è usato dal metodo Valida..

IsDisabilitata

Restituisce se la proprietà è disabilitata. E' considerato un parametro legato alla logica video quindi va usato solo soltanto dal metodo ImpostaAttributiVideo().
Per questo motivo va impostato con VER=1, in modo che non serva indicarlo nelle chiamate da logica applicativa.

Gestita

Valore di ritorno che indica se la proprietà è gestita o meno.

Esempio

```
BC

'@SBP This[CLASSE[]] [INOUT] _
      ProprietaCorrente[STRING] [IN] _ ' Proprieta corrente
      Errore[CLASSE[BC_VID_ERROR]] [IN] _ ' Errore riscontrato
      IsDisabilitata[TIPO[BCBOOL]] [OUT] [VER=1] _ ' Proprieta disabilitata (da usare nell'
ImpostaAttributiVideo)
      AS Gestita[TIPO[BCBOOL]]

Gestita = #True

Switch ProprietaCorrente
  Case This.PROP1.PropertyId()
    If Not This.PROP2 And This.PROP3 = "" Then Gestita = #False
  Case ...
EndSwitch

End
```

Riferimenti

Metodo ImpostaAttributiVideo

IsProprietaProtetta

Contiene la **logica applicativa** che determina se una proprietà è protetta da modifiche o meno.
Il suo scopo è quello di **centralizzare** tale logica sia per uso UI che BL:
In ambito UI viene richiamato dal metodo ImpostaAttributiVideo() per sapere se impostare l'attributo #OUTPUT.

In ambito BL viene richiamato per sapere se è una proprietà è protetta in quel contesto, ad esempio in fase di validazione o derivazione dati.

Firma

Definizione BC			
'@SBP	This[CLASSE[<classe>]]	[INOUT]	_
	ProprietaCorrente[STRING]	[IN]	_
	Errore[CLASSE[BC_VID_ERROR]]	[OUT]	_
AS Protetta[TIPO[BCBOOL]]			

Parametri

This
Oggetto sul quale impostare gli attributi.

ProprietaCorrente
Id della proprietà corrente per la quale valutare l'impostazione degli attributi. Va confrontato con il PropertyId di ogni proprietà. Se blank, richiama l'impostazione degli attributi di tutte le proprietà.

Errore
Eventuale errore che indica il motivo per il quale la proprietà è protetta.
L'errore viene emesso da questo metodo perché deve essere il più possibile parlante.
L'errore è usato dal metodo AssegnaDati per importazioni e servizi.

Protetta
Valore di ritorno che indica se la proprietà è protetta oppure no.

Esempio

```
BC

'@SBP This[CLASSE[]] [INOUT] _
  ProprietaCorrente[STRING] [IN] _ ' Proprieta corrente
  Errore[CLASSE[BC_VID_ERROR]] [OUT] _ ' Errore riscontrato
  AS Protetta[TIPO[BCBOOL]]

Switch ProprietaCorrente
  Case This.PROP1.PropertyId()
    If Not This.PROP2 And This.PROP3 = "" Then Protetta = #True
  Case ...
EndSwitch

End
```

Riferimenti

Metodo ImpostaAttributiVideo

IsUtilizzabile

Metodo che carica l'entità e controlla se è utilizzabile.
All'interno di questo metodo vanno inseriti tutti i controlli che devono sempre essere fatti per determinare se un'entità è utilizzabile o meno.
Possono esistere anche altri metodi IsUtilizzabileXXX più specifici e con controlli aggiuntivi legati al contesto.
Il metodo è statico perché può essere utilizzato in contesti in cui non ci serve avere l'istanza dell'oggetto.
Il metodo può prevedere parametri aggiuntivi che servono ad effettuare i vari controlli (ad es. la data per un controllo sulla data di validità).

Firma

Definizione BC			
'@SBP	Chiave	[IN]	_
	Errori[Classe[BC_VID_ERROR] COLLEZIONE]	[OUT]	_
	Oggetto[CLASSE[]]	[OUT]	_
	AS IsUtilizzabile[TIPO[BCBOOL]]		

Parametri

This

Oggetto da caricare.

Chiave

Uno o più parametri che rappresentano i campi chiave.

Errori

Collezione di eventuali errori riscontrati nei vari controlli

Oggetto

Istanza dell'oggetto caricato e controllato

IsUtilizzabile

Valore di ritorno che indica se l'oggetto è utilizzabile

BC			
<pre>'@SBP Chiave [IN] _ ' Chiave Errori[CLASSE[BC_VID_ERROR] COLLEZIONE] [OUT] _ ' Errori Oggetto[CLASSE[%Classe%]] [OUT] _ ' Oggetto caricato e controllato AS IsUtilizzabile[TIPO[BCBOOL]] IsUtilizzabile = Oggetto.Carica(Chiave, 0, #True, Errori [OUT]) ' Controlli aggiuntivi End</pre>			

QueryElenco

Il metodo QueryElenco va fatto sulla classe e deve essere un metodo **statico**, nel quale viene definita la query da utilizzare nel metodo Elenco.

Non va mai chiamato direttamente, se non dal metodo Elenco() (e ElencoWeb o ElencoMobile).

Firma

Definizione BC			
'@SBP	IdQueryElenco[INT]	[IN]	_
	MultiSel[TIPO[BCBOOL]]	[IN]	_
	CodPreSelezionati[TIPO[BCSTRING]]	[IN]	_
	EscludiFiltri[TIPO[BCBOOL]]	[IN]	_
	Filtri	[INOUT]	

Parametri

IdQueryElenco

Id della query da definire. La '@DEFQUERY' deve essere fatta nel chiamante.

[Multisel]

Indica se gestire la multi selezione. Non è necessario se non si intende gestirla.

[CodPreSelezionati]

Stringa di ingresso contenente l'elenco dei codici (separati da una virgola) selezionati ad ingresso videata.

Se la griglia è caricata da query, questa stringa va usata per impostare un campo calcolato in query con alias ZSEL, che servirà a selezionare le righe in griglia.

Non è necessario se non si gestisce la multi selezione.

EscludiFiltri

Se valorizzato indica di non definire i filtri della query. Viene passato a #TRUE dal metodo [QueryElencoAggiornamento\(\)](#).

Filtri

Elenco di filtri da applicare.

Può essere una classe applicativa di filtri o un elenco di parametri semplici.

Riferimenti

Metodo [Elenco](#)

Metodo [QueryElencoAggiornamento](#)

QueryElencoAggiornamento

Il metodo QueryElencoAggiornamento() va fatto sulla classe e deve essere un metodo **statico**.

Serve per ridefinire la query dell' [Elenco\(\)](#) in modo da effettuare l'aggiornamento a seguito di un'aggiunta o modifica di un record dell'elenco.

Firma

Definizione BC		
'@SBP	IdQueryElenco[INT]	[IN] _
	Chiave	[IN]

Parametri

IdQueryElenco

Id della query da definire. La '@DEFQUERY' deve essere fatta nel chiamante.

Chiave

Elenco di parametri che rappresentano la chiave dell'entità aggiunta o modificata.

Esempio

BC
<pre>'@SBP IdQueryElenco[INT] [IN] _ Chiave[INT] [IN] <Classe>.QueryElenco(IdQueryElenco, #True, #False, "") '@DEFQUERYWHERE IDQUERY[IdQueryElenco] ESPR[Tabella.Chiave=Chiave] End</pre>

Riferimenti

Metodo [Elenco](#)

Metodo [QueryElenco](#)

Sblocca

Metodo statico che sblocca la variazione dell'entità in seguito ad un blocco fatto mediante il metodo [Blocca](#).

Firma

Definizione BC			
'@SBP	Lock[CLASSE[BC_DA_APPLOCK]]	[INOUT]	

Parametri

Lock

Oggetto che contiene le informazioni del lock precedentemente acquisito.

BC	
<pre>'@PG BLD_FATTURA_SBLOCCA STRICT[3] ' Sblocco entita '@SBP Lock[CLASSE[BC_DA_APPLOCK]] [INOUT] ' Oggetto del lock da rilasciare Lock.ReleaseAppLock() End</pre>	

Valida

E' il metodo che si occupa di validare l'entità e impostare eventuali errori su proprietà/oggetti.

Per la validazione dell'intero oggetto (ProprietaCorrente vuota) va usato il metodo di ambiente [ValidateFromClass\(\)](#) prima delle specifiche validazioni delle proprietà.

Prima di chiamare il metodo [ValidateFromClass\(\)](#) deve essere richiamato il metodo applicativo [ImpostaAttributoObbligatorio](#).

La validazione della singola proprietà non setta l'errore se il codice non è valorizzato.

In questo modo l'utente non viene vincolato nella navigazione della videata ma gli errori vengono dati a conferma, ovvero alla validazione dell'intero oggetto.

In caso di oggetti composti il metodo deve eseguire la validazione di tutti i componenti oggetti e collezioni (solo se sono [IsModified\(\)](#)).

Nella validazione complessiva, come ultima operazione, eseguire senza condizionamenti sullo stato di validazione i controlli sulla chiave primaria della tabella di riferimento dell'entità. Questo per eseguire il controllo di univocità della chiave subito prima di eseguire l'aggiornamento dei dati su database.

In caso di controlli incrociati che coinvolgono più proprietà, essi vanno controllati nella validazione complessiva (ProprietaCorrente vuota) e l'errore va impostato sull'oggetto. Per questi errori impostati sull'oggetto è importante assegnare un codice univoco (vedi scheda [Enumerato Errori](#)) perché devono essere rimossi nella validazione delle proprietà che impattano su quell'errore.

Firma

Definizione BC			
'@SBP	This[CLASSE[<classe>]]	[INOUT]	_
	ProprietaCorrente[STRING]	[IN]	

Parametri

This

Oggetto da validare

ProprietaCorrente

Id della proprietà corrente da validare. Va confrontato con il PropertyId di ogni proprietà per capire quale validare. Se blank, va validato l'intero oggetto.

Esempio

BC

```
'@SBP This[CLASSE[]] [INOUT] _ ' Oggetto da validare
    ProprietaCorrente[STRING] [IN] ' Proprietà corrente (Per validazione singola
proprietà)

    If ProprietaCorrente="" Then

        This.ResetError(0)
        This.ImpostaAttributoObbligatorio("")
        This.ValidatefromClass()

'#region _Autogen_Valida_If
    If This.CAMPO1.IsToValidate() Then ValidaCAMPO1()
'#endregion

    ' La validazione della proprietà che identifica la chiave univoca dell'entità va eseguita sempre,
senza testare lo stato di validazione
    If Not This.HasError(#ERRORE) Then ValidaCodice()

    ' La validazione delle proprietà composte (oggetti e collezioni) va fatta soltanto se esse sono
modificate, in modo da non caricare tutta l'entità
    If This.Componente.IsModified() Then This.Componente.Valida("")

    Else

'#region _Autogen_ValidaDeriva_Switch
    Switch ProprietaCorrente

        Case This.CAMPO1.PropertyId() ValidaCAMPO1()

    EndSwitch
'#endregion

    EndIf

End

'@SRP ValidaCAMPO1()

    ' Eliminazione errori per la proprietà
    This.CAMPO.ResetError()

    If This.CAMPO="" Then Exitsr

    If ENTITA.Carica(This.CAMPO [IN], 0 [IN]) = 0 Then
        This.CAMPO.SetError(#ERRORE,1,"Il Codice Entita inserito non esiste.")
        Exitsr
    EndIf

    This.CAMPO.SetValidationState(#VALIDO)

Return
```

Riferimenti

Metodo [Deriva](#)

Metodo [ImpostaAttributoObbligatorio](#)

Enumerato [Errori](#)

Estendere la validazione di proprietà di una sottoclasse

In determinati contesti può essere necessario estendere le logiche di validazione di alcune proprietà di una sottoclasse.

Un caso di esempio è la classe video, per la quale può essere necessario estendere la validazione di una proprietà del modello dati in funzione di logiche specifiche della videata.

Nel fare questa operazione bisogna tenere conto di alcuni accorgimenti necessari a far funzionare correttamente il giro di validazione.

Di seguito un esempio del flusso di validazione da impostare per ottenere un corretto funzionamento.

Nell'esempio viene simulato di dover estendere, sulla classe video, la validazione di alcune proprietà del modello dati.

Blocco principale del metodo Valida():

BC
<pre>If ProprietaCorrente="" Then This.ResetError(0) This.ImpostaAttributoObbligatorio("") ValidaModelloDati() This.ValidateFromClass() If This.IsToValidate() And Not This.HasError(#ERRORE) Then This.ControlliFinali() If Not This.HasError(#ERRORE) Then This.SetValidationState(#VALIDO) Else ValidaModelloDati() EndIf</pre>

Validazione sottoclasse specifica (in questo caso il modello dati):

BC

```

'@SRP ValidaModelloDati ()

If ProprietaCorrente = "" Then

    ' Blocco validazione proprietà da estendere
    ' Il test su IsToValidate non va messo
    ValidaModelloDati_CodTipoDoc()
    ValidaModelloDati_DataRegistrazione()

    ' Richiamo la validazione standard per le proprietà da non estendere
    This.ModelloDati.Valida("")

Else

    Switch ProprietaCorrente

        ' Blocco validazione proprietà da estendere
        Case This.ModelloDati.CodTipoDoc.PropertyId()
            ValidaModelloDati_CodTipoDoc()

        Case This.ModelloDati.DataRegistrazione.PropertyId()
            ValidaModelloDati_DataRegistrazione()

        ' Richiamo la validazione standard per le proprietà da non estendere
        Case Else
            This.ModelloDati.Valida(ProprietaCorrente)

    EndSwitch
EndIf

Return

```

Flusso di validazione della singola proprietà:

BC

```

'@SRP ValidaModelloDati_CodTipoDoc() ' Estensione validazione proprietà CodTipoDoc

' Elimino gli errori sulla proprietà (potrebbe non essere presente la validazione della proprietà sulla
classe specifica)
This.ModelloDati.CodTipoDoc.ResetError()

' Richiamo la validazione sulla classe specifica
This.ModelloDati.Valida(This.ModelloDati.CodTipoDoc.PropertyId())

' Se la validazione della classe specifica ha impostato un errore esco
If This.ModelloDati.CodTipoDoc.HasError(#ERRORE) Then ExitSr

' Se la proprietà non è valorizzata esco
If This.MoidelloDati.CodTipoDoc = 0 Then ExitSr

' Codice di validazione personalizzato
...

' Imposto la proprietà come valida
This.ModelloDati.CodTipoDoc.SetValidationState(#VALIDO)

Return

```


Riferimenti

Estendere la derivazione di proprietà di una sottoclasse

Metodi di controllo

Per definire il nome di metodi che servono a controllare un particolare stato dell'oggetto o determinare se si può eseguire una specifica azione, va usato il prefisso "Is".

Esempi: IsEliminabile, IsStampabile, IsRiclassificabile, ecc.

Per definire il nome di metodi che servono a controllare se l'oggetto "contiene" qualcosa va usato il prefisso "Has".

Esempi: HasBilanciRiclassificati, "HasRigheEvadibili", ecc.

Firma

Definizione BC			
'@SBP	This[CLASSE[<classe>]]	[INOUT]	_
	Parametri	[IN]	_
	Errori[Classe[BC_VID_ERROR] COLLEZIONE]	[OUT]	_
AS Valido [TIPO[BCBOOL]]			

Parametri

This

Oggetto, già caricato

Parametri

Elenco di parametri utili al controllo da effettuare

Errori

Collezione di errori riscontrati dal controllo

Valido

Valore di ritorno che indica se il controllo è andato a buon fine

Attributi delle proprietà

L'attributo di obbligatorietà (#REQUIRED) è l'unico da intendersi come attributo valevole sia per l'interfaccia utente che per la logica applicativa, principalmente per la validazione dell'entità.

Tale attributo va impostato nel metodo applicativo ImpostaAttributoObbligatorio, che deve essere chiamato dal metodo Valida() e dal metodo ImpostaAttributiVideo.

***** Quando lo stato di obbligatorietà della proprietà è statico utilizzare l'impostazione a livello di definizione della classe. Questa viene automaticamente gestita sia a video che in validazione con il metodo Validatefromclass().

Tutti gli attributi sono da intendersi ad uso **esclusivo** dell'interfaccia utente e vanno impostati nel metodo ImpostaAttributiVideo.

Per centralizzare la logica applicativa che serve sia ad impostare l'attributo #HIDDEN che ad indicare che una proprietà non è gestita, va fatto il metodo IsProprietaGestita().

Tale metodo va richiamato nel metodo ImpostaAttributiVideo e può essere anche richiamato dalla logica applicativa per determinare se una proprietà è gestita o meno.

Discorso analogo vale per l'attributo #OUTPUT e il metodo IsProprietaProtetta().

Riferimenti

Metodo ImpostaAttributoObbligatorio

Metodo ImpostaAttributiVideo

Metodo IsProprietaGestita

Metodo IsProprietaProtetta

Set/Get Attribute

Tipo dato

- [Metodi](#)

Metodi

Metodo	Contesto	Tipo	Validità	Descrizione
GetVarVideo	Oggetto	BL	Multipiattaforma	Restituisce il VarVideo. Opzionale in base alla presenza di filtri per la query di autocompletamento, altrimenti il varvideo viene gestito in automatico dal tipo dato.
GetVarVideoMultisel	Statico	BL	Multipiattaforma	Restituisce il VarVideo e il commento.. Opzionale in base alla necessità di gestire la multiselezione sull'entità
Ricerca	Oggetto	BL	Desktop	Metodo wrapper che richiama il metodo Elenco della classe in modalità Ricerca o RicercaConGestione
RicercaWeb	Oggetto	UI	Web	Videata di ricerca Web
RicercaMobile	Oggetto	UI	Mobile	Videata di ricerca Mobile
RicercaMultisel	Statico	BL	Desktop	Metodo wrapper che richiama il metodo Elenco della classe in modalità Ricerca o RicercaConGestione, attivando la multiselezione. Opzionale in base alla necessità di gestire la multiselezione sull'entità
QueryAutocompletamento	Statico	BL	Desktop	Definizione query di autocompletamento
QueryAutocompletamentoWeb	Statico	BL	Web	Definizione query di autocompletamento Web
QueryAutocompletamentoMobile	Statico	BL	Mobile	Definizione query di autocompletamento Mobile

GetVarVideo

Il metodo GetVarVideo() deve essere un metodo del tipo dato.
E' il metodo che si occupa di decodificare il VarVideo partendo dall'Id.

La creazione di tale metodo è necessaria soltanto se non è possibile creare la decodifica da tipo dato (ad esempio perché la chiave della tabella è composta).
In tutti gli altri casi, creando la decodifica dal tipo dato, il VARVIDEO non è necessario e quindi nemmeno il metodo GetVarVideo.

Firma

Definizione BC	
'@SBP	This[TIPO[<TipoDato>]] [IN] _
AS VarVideo[STRING]	

Parametri

This
Stringa contenente l'elenco di codici selezionati (separati da una virgola).

VarVideo
Valore di ritorno che conterrà il VarVideo.

Riferimenti

Metodo [GetVarVideo](#)

GetVarVideoMultisel

Il metodo GetVarVideoMultisel() deve essere un metodo statico del tipo dato.
E' il metodo che si occupa di restituire il VarVideo decodificando i codici selezionati.

La creazione di questo metodo va fatta solo quando è necessario gestire la multiselezione sull'entità.

Firma

Definizione BC			
'@SBP	CodSelezionati[STRING]	[IN]	-
	Commento[STRING]	[OUT]	-
	AS VarVideo[STRING]		

Parametri

CodSelezionati

Stringa contenente l'elenco di codici selezionati (separati da una virgola).

Commento

Stringa che conterrà la descrizione dei codici selezionati, da usare come tooltip in videata.

Se sono selezionati più di 5 valori, va indicata una stringa generica per evitare di appesantire troppo la query a ingresso videata.

VarVideo

Valore di ritorno alfanumerico che conterrà il VarVideo dei codici selezionati. Se è selezionato più di un valore va restituito [...].

Esempio

Riferimenti

Metodo [GetVarVideo](#)

QueryAutocompletamento

È il metodo che centralizza la query da usare per l'autocompletamento.

È un metodo del tipo dato perché può essere richiamato in automatico dalla proprietà agganciata al campo di edit.

Può anche essere chiamato manualmente quando è necessario passare dei filtri alla query.

Firma

Definizione BC			
'@SBP	IdQuery[INT]	[IN]	
	Filtri	[INOUT]	[VER=1]
	SoloCodice[TIPO[BCBOOL]]	[IN]	[VER=1]

Parametri

IdQuery

Id della query. La '@DEFQUERY va fatta nel sorgente chiamante.

Filtri

Elenco di filtri da applicare.

Può essere una classe applicativa di filtri o un elenco di parametri semplici. Deve essere versionato perché nel richiamo automatico d'ambiente non è previsto.

SoloCodice

Flag che indica se estrarre solo la chiave nel VARVIDEO.

Se spento non viene passato a True, di default il VARVIDEO deve essere composto da Codice - Descrizione.

Deve essere versionato perché nel richiamo automatico d'ambiente non è previsto.

Ricerca

Il metodo Ricerca è un metodo del tipo dato perché viene richiamato in automatico dalla proprietà agganciata al campo di edit.

Si tratta di un metodo "wrapper" che deve richiamare il metodo statico Elenco(), previsto sulla classe, nel quale è centralizzata la videata.

Può essere richiamato manualmente se necessario passare dei filtri.

Firma

Definizione BC					
'@SBP	This[TIPO[<TipoDato>]]	[IN]			-
	AbilitaGestione[TIPO[BCBOOL]]	[IN]	[VER=1]	[DEFAULT=#False]	-
	Filtri	[INOUT]	[VER=1]		-
AS ValoreSelezionato[TIPO[<Tipo>]]					

Parametri

AbilitaGestione

Flag che indica se attivare, nella ricerca, i pulsanti per la gestione dell'entità (Varia/Elimina/ecc).

Il parametro deve essere versionato perché potrebbe non esserci e l'ambiente, nel richiamo automatico della sub di ricerca, non se lo aspetta.

Filtri

Elenco di filtri da applicare.

Può essere una classe applicativa di filtri o un elenco di parametri semplici. Deve essere versionato perché nel richiamo automatico d'ambiente non è previsto.

ValoreSelezionato

Valore di ritorno selezionato nella videata di ricerca.

Esempio

BC	
<pre>'@SBP This[TIPO[<Tipo>]] [IN] - AbilitaGestione[TIPO[BCBOOL]] [IN] [VER=1] [DEFAULT=#False] - DataValidita[DATE] [IN] [VER=1] - AS ValoreSelezionato[TIPO[<Tipo>]] DIM Mode[TIPO[BCTD_MODALITAELENCO]] If AbilitaGestione Then Mode = BCTD_MODALITAELENCO.RicercaConGestione Else Mode = BCTD_MODALITAELENCO.Ricerca Endif Classe.Elenco(Mode, #False, DataValidita, This, "", CodSelezionato[TIPO[BCSTRING]] [OUT]) If CodSelezionato <> "" Then ValoreSelezionato = CodSelezionati ' Assegno il valore selezionato e converto se necessario Else ValoreSelezionato = This Endif End</pre>	

Riferimenti

Metodo Elenco

Metodo RicercaMultisel

RicercaMultisel

Il metodo RicercaMultisel deve essere un metodo statico del tipo dato.
Si tratta di un metodo "wrapper" che deve richiamare il metodo statico Elenco(), previsto sulla classe, nella quale è centralizzata la videata.

La creazione di questo metodo va fatta solo quando è necessario gestire la multiselezione sull'entità.

Firma

Definizione BC			
'@SBP	CodPresel[STRING]	[IN]	—
	Filtri	[IN/INOUT]	—
	AbilitaGestione[TIPO[BCBOOL]]	[IN]	—
	AS CodSelezionati[STRING]		

Parametri

CodPresel
Stringa contenente l'elenco di codici pre-selezionati (separati da una virgola).

Filtri
Elenco di filtri da applicare.
Può essere una classe applicativa di filtri o un elenco di parametri semplici.

AbilitaGestione
Flag che indica se attivare, nella ricerca, i pulsanti per la gestione dell'entità (Varia/Elimina/ecc).

CodSelezionati
Stringa che conterrà l'elenco di codici selezionati nella ricerca con multi selezione.

Esempio

BC	
<pre>'@SBP CodPresel[STRING] [IN] — DataValidita[DATE] [IN] — AbilitaGestione[TIPO[BCBOOL]] [IN] — AS CodSelezionati[STRING] DIM Mode[TIPO[BCTD_MODALITAELENCO]] If AbilitaGestione Then Mode = BCTD_MODALITAELENCO.RicercaConGestione Else Mode = BCTD_MODALITAELENCO.Ricerca Endif Classe.Elenco(Mode, #True, DataValidita, "", CodPresel, CodSelezionati [OUT]) If CodSelezionati = "" Then CodSelezionati = CodPresel End</pre>	

Riferimenti

- Metodo Elenco
- Metodo Ricerca

Classi di filtri

Sono le classi da creare per la gestione di filtri.
La loro nomenclatura prevede il suffisso `_FILTRI`.

Possono essere usate:

- Per la definizione delle videate di filtri
- Come parametro dell'oggetto query del metodo Elenco e dei metodi Gestione, Ricerca, RicercaMultisel

Classi Helper

Le classi helper sono classi statiche che non hanno proprietà e contengono solo metodi statici. Pertanto non possono essere istanziate.

Quando definirle

Le classi helper vanno fatte per raggruppare sotto un contesto comune un insieme di sub che altrimenti non sarebbero utilizzabili come metodi.

Come standard abbiamo definito che le classi helper non vanno fatte per le entità che rappresentano un modello dati. Se devo definire un metodo legato ad un'entità ma che non ha bisogno dei dati dell'entità stessa, esso va definito sul tipo dato, se è correlato ad esso, altrimenti come metodo statico sulla classe dell'entità. Per maggiori dettagli vedere [Logica di definizione di un metodo](#).

Come definirle

Devono essere classi statiche aventi come nome il contesto seguito dal suffisso `_HELPER`.

Enumerato Errori

Al fine di identificare i codici di errore che devono essere rimossi o ricercati tra gli errori dell'entità, è opportuno creare un apposito enumerato di errori in modo da evitare di spargere costanti numeriche nei sorgenti.

Come definirlo

L'enumerato deve seguire la nomenclatura `<Sigla prodotto>TD_<Entita>_ERRORI` e deve contenere un valore per ogni codice di errore che dovrà essere identificato.

Per convenzione e compatibilità con il pregresso i codici numerici degli enumerati devono partire da 100.

Quando definirlo

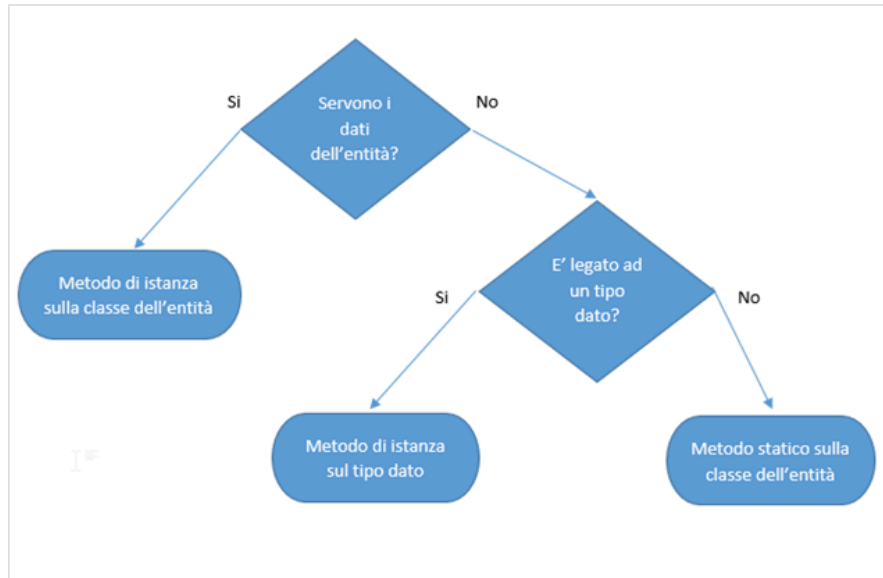
L'enumerato va definito soltanto se ci sono codici di errore che devono essere rimossi o ricercati tra gli errori dell'entità: ad esempio errori che coinvolgono più proprietà e devono rimossi nella validazione delle singole proprietà o errori che devono essere rimossi a fronte di un'azione a video.

Esempio di utilizzo nel Valida

BC
<pre>'@SRP ValidaFinale() This.RemoveError(ES_TD_CONDPAGAMENTO_ERRORE.DesLinguaNonIndicate) If This.Des = "" And This.DesEstesa = "" Then This.SetError(#ERRORE, ES_TD_CONDPAGAMENTO_ERRORE.DesLinguaNonIndicate, "Indicare un valore per " + This.Des.Label().Trim() + " o " + This.DesEstesa.Label().Trim()) EndIf Return</pre>

Logica di definizione di un metodo

Per capire dove e come definire un metodo va seguito questo schema logico:



Interfaccia Utente

- [Modello video](#)
- [Impostazione stati video dei controlli](#)
- [Gestione progress bar](#)
- [Videate con sottoinsieme di proprietà del modello](#)
- [Quale griglia utilizzare e quando](#)

Modello video

La classe video, o modello video, è una classe che rappresenta i dati in modo preciso rispetto a come vanno esposti a video.

Quando definirla

La classe video va definita soltanto quando il modello dati differisce da come vanno esposti i dati a video o quando è necessario integrare i dati di informazioni aggiuntive utili alla rappresentazione a video.

Scenari possibili

1. Informazioni aggiuntive in input su griglia o videata → Si
2. Esposizione in videata di un solo oggetto di una collezione contenuta nel modello dati → Dipende dal contesto (vedi paragrafo Come definirla).
3. Presenza di dati in output a corredo dell'entità (classiche decodifiche) → No, esse possono essere gestite con variabili locali valorizzate nelle callback DECOD degli EDIT.
4. Colonne aggiuntive di solo output su griglie → No, vanno usate lookup o colonne calcolate.

Cosa comporta gestirla

La presenza della classe video può comportare alcuni oneri:

- Dover caricare tutta l'entità perdendo i vantaggi del lazy loading
- In caso di collezione, va mantenuta allineata con i dati facendo le aggiunte e le rimozioni su entrambe le collezioni

Come definirla

💡 Utilizza la funzione "Genera classe video" presente in AGIS. Ulteriori dettagli in [questa scheda](#).

Prodotti non personalizzabili

Il nome della classe video deve essere uguale a quello della rispettiva classe dati con il suffisso "_VID_BASE".

Potenzialmente potrebbero esistere più modelli video per un'entità oppure modelli video specifici per il contesto di riferimento (Desktop, Web, Mobile) se vi sono differenze nella loro rappresentazione.

In tal caso, tali classi avranno un nome specializzato "_VID_<contesto>" (es. _VID_DESKTOP) e avranno come base la classe _VID_BASE, che conterrà solo le proprietà e i metodi comuni ai vari contesti.

Prodotti personalizzabili

Per i prodotti personalizzabili il pattern indicato in precedenza è sconsigliato perché quando si differenziano le classi video per i vari contesti è necessario rifattorizzare la classe _VID_BASE e questo potrebbe rompere la compatibilità con le personalizzazioni.

Pertanto, per i prodotti personalizzabili, va sempre creata la classe video specifica per il contesto, evitando la classe base.

Ad esempio se il partner ha personalizzato il metodo "Carica()" quando associato alla classe VID_BASE, nel momento in cui viene fatta la classe VID_DESKTOP la personalizzazione può saltare.

Nel caso semplice si può garantire richiamando, dal nuovo metodo Carica(), il vecchio metodo della classe Classe_VID_Base. Ma potrebbero esserci situazioni più complesse da gestire. Soprattutto se si fa il popolamento delle proprietà video tutto dentro il Carica() della classe video. In quel caso si sposta solo parte del codice e garantire la compatibilità con le personalizzazioni può essere un problema.

Scenario 1

Per lo scenario 1, ovvero estensione del modello dati, la classe video base avrà come prima proprietà, con nome ModelloDati, la classe dati più le proprietà aggiuntive da esporre a video.

La proprietà ModelloDati dell'oggetto video deve puntare per riferimento all'oggetto della classe dati, in modo che le modifiche fatte all'uno si ribaltino sull'altro.

Scenario 2

Per lo scenario 2, se l'oggetto è da rappresentare senza altri dati in un multipagina, si può usare direttamente, come modello della pagina, la classe della collezione contenuta nel modello dati.

Se invece l'oggetto è da rappresentare con altri dati (ad esempio i dati di testata) allora va fatto un modello video contenente gli altri dati più una proprietà oggetto dello stesso tipo della collezione contenuta nel modello dati.

Se serve fare il modello video su una pagina del multipagina si gestisce il modello video sulla specifica videata della pagina, andando in override rispetto al modello dati dell'intero multipagina.

In tal caso però la validazione complessiva del modello di pagina non viene mai chiamata, soltanto quella delle singole proprietà a uscita campo.

Modello video su multipagina

In caso di modello video usato come modello del multipagina, i metodi che rappresentano le varie pagine vanno creati solo sul modello video.

Il modello video del multipagina deve contenere soltanto la classe dati.

Se sono necessari altri modelli video per le altre pagine, essi vanno istanziati direttamente nel sorgente della pagina.

Vincoli

Le classi video **NON vanno rese pubbliche**. Visto che potenzialmente possono essere scomposte in base alle esigenze.

E' il modello dati che viene pubblicato e da esso parte il metodo Dettaglio() (o analoghi) che al loro interno definisce e carica la classe video.

Esempio

Ipotizziamo si parta dall'esigenza di dover definire la videata desktop che necessita di classe video.

Si parte quindi dalla classe base _VID_BASE:



Successivamente nasce l'esigenza di avere l'esposizione Web, ma in modo diverso da Desktop. Viene quindi definita la classe video per il Web e di conseguenza anche la classe video Desktop, che conterrà le proprietà esclusive per il suo contesto. Per farlo si passa a rifattorizzare la definizione della classe base, lasciando solo ciò che è in comune ai due contesti:



Di conseguenza le classi video desktop e web



Passi operativi per arrivare a questa definizione:

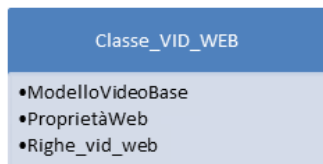
- Creazione nuove classi "Classe_VID_DESKTOP" e "Classe_VID_WEB" come copia dalla classe "Classe_VID_BASE"
 - Si possono scegliere i metodi da copiare e vengono quindi create le nuove sub per le nuove classi
 - Eliminazione dalle nuove classi delle proprietà che restano sulla classe base
 - Aggiunta della proprietà base di tipo Classe_VID_BASE
 - Aggiunta delle proprietà specifiche
- Eliminazione dalla classe "Classe_VID_BASE" delle proprietà che diventano specifiche per le classi desktop/web
- Adeguamento dei metodi della classe base, che non devono più trattare le proprietà specializzate
- Adeguamento dei metodi delle nuove classi. Richiamo del Carica della classe "Classe_VID_Base".

Per le successive esigenze di ulteriori proprietà sui modelli video va valutato se queste possono essere comuni ai due contesti o meno.

Se lo sono si estende la classe "Classe_VID_Base".

Se non lo sono va estesa la specifica classe.

Ipotizziamo di estendere la classe "Classe_VID_Web" con una collezione di righe



Nel momento in cui nasce l'esigenza di impostare la videata per il mobile va ragionato come intervenire nella definizione delle classi video.

Se la classe video "Classe_VID_Base" ha elementi comune anche per l'ambito Mobile la classe "Classe_VID_Mobile" conterrà la base più le proprietà specifiche.

Se invece nella base ci sono proprietà che non servono all'ambito mobile ci sono due strade:

- Si estrae dalla base le proprietà che non servono al mobile e si portano su classe video desktop e web. In questo caso vanno adeguati i metodi duplicandoli sulle due classi
- Si mantiene la classe "Classe_VID_Base" inalterata e la videata mobile non userà alcune proprietà.

Se il caricamento delle proprietà inutilizzate dalla videata mobile non è rilevante in termini di operazioni eseguite si può seguire la seconda strada. **NON vanno però gestiti test sull'ambito di esecuzione per condizionare il codice.**

L'oggetto deve essere consistente, non può avere proprietà non popolate solo perché in alcuni ambiti non sono rilevanti.

Impostazione stati video dei controlli

Per gestire gli stati video dei controlli (nascosto, disabilitato, output) si possono usare sia gli attributi impostabili sulle proprietà dell'oggetto (tramite `SetAttribute`) che i vari parametri NASCOSTO/DISABILITATO/ecc dei controlli video.

Quando uno stato è definito da una condizione presente nei dati, allora va impostato tramite l'attributo sulla proprietà dell'oggetto (vedi scheda [Attributi delle proprietà](#)). Un esempio può essere la modifica di una proprietà che ne attiva un'altra. Nel metodo `Deriva()` della prima va modificato l'attributo sulla seconda.

Quando invece è definito da un'azione compiuta a video oppure si tratta di un campo non presente nel modello, allora va impostato tramite i parametri presenti sulla specifica video. Un esempio può essere l'azione di un tasto che abilita la variazione dei dati. È un elemento video non è relazionato ai dati.

Riferimenti

Metodo [ImpostaAttributiVideo](#)

Metodo [ImpostaAttributoObbligatorio](#)

Gestione progress bar

Per dare evidenza all'utente che è in corso un'elaborazione è possibile utilizzare una progress bar. Quest'ultima può essere di tre tipologie:

- Avanzamento "generico"
- Avanzamento unica fase con dettagli
- Avanzamento a fasi con eventuali dettagli

La scelta di quale usare dipende molto dal contesto e dall'esigenza applicativa di dare o meno evidenza delle fasi/dettagli che l'elaborazione sta eseguendo.

Dal punto di vista di architettura standard i metodi di business che eseguono **l'elaborazione non devono contenere** la definizione del controllo grafico che emette la progress bar. Questa nell'ottica di sviluppo multiplatforma è ancora più importante che sia seguita come indicazione.

Dato per assodato quanto indicato in precedenza, di seguito le indicazioni per gestire le diverse modalità di applicazione di una progress bar in modo standard.

Progress bar avanzamento generico

Utilizzare la '@DEFVIDWAITMSG con `VISAVANZAMENTO[2]` direttamente prima della chiamata al metodo che esegue l'elaborazione. Tipicamente sarà quindi definita nell'azione richiamata dalla videata principale che avvia l'elaborazione

Progress bar con fasi/dettagli

Utilizzare la '@DEFVIDPROGRESSBAR definendo l'oggetto di tipo `BC_VID_PROGRESS` in base al dettaglio che si vuole dare all'utente. L'oggetto viene passato in firma ai metodi di business che eseguono l'elaborazione. La definizione della progress bar va fatta direttamente prima della chiamata al metodo che esegue l'elaborazione. Tipicamente sarà quindi definita nell'azione richiamata dalla videata principale che avvia l'elaborazione. Se l'elaborazione viene richiamata come web service non ci sarà ovviamente un riscontro video. L'oggetto con l'avanzamento delle fasi dell'elaborazione può eventualmente essere restituito o trattato dal chiamante come traccia(log) delle operazioni eseguite.

Videate con sottoinsieme di proprietà del modello

Per alcune videate di dettaglio applicativamente viene scelto di spostare determinati campi meno importanti in una sottovideata.

Per queste videate non va usato come modello dati la stessa classe usata nella videata principale.

Va invece fatta una classe video specifica con i soli campi presenti nella videata.

Su tale classe devono essere creati i metodi Valida() e Deriva(), che vanno impostati come di seguito.

Valida()

Il Valida() si limita a clonare i dati dalla classe video alla classe dati e successivamente richiamare il Valida() della classe dati passando il rispettivo PropertyId.

Dopo averlo chiamato devono essere copiati eventuali errori e lo stato di validazione dalla classe dati a quella video:

BC
<pre>'@SBP This[CLASSE[CLASSE_VID]] [INOUT] _ Dati[CLASSE[CLASSE_DATI]] [INOUT] _ ProprietaCorrente[STRING] [IN] ' Proprietà corrente ' Copia dei valori da classe video a classe dati This.Clone(Dati, 1) If ProprietaCorrente = "" Then This.ValidateFromClass() If This.PROP1.IsToValidate() Then Prop1() If This.PROP2.IsToValidate() Then Prop2() If This.PROP3.IsToValidate() Then Prop3() ' Inserire qui eventuali controlli finali Else Switch ProprietaCorrente Case This.PROP1.PropertyId() Prop1() Case This.PROP2.PropertyId() Prop2() Case This.PROP3.PropertyId() Prop3() EndSwitch EndIf End '@SRP Prop1() This.Prop1.ResetError() Dati.Valida(Dati.Prop1.PropertyId()) This.Prop1.SetErrors(Dati.Prop1.GetErrors()) This.Prop1.SetValidationState(Dati.Prop1.GetValidationState()) Return</pre>

Deriva()

Il Deriva() si limita a richiamare il Deriva() della classe dati e in seguito a clonare la classe dati sulla classe video per riportare le modifiche fatte dalla derivazione.

BC

```
'@SBP This[CLASSE[CLASSE_VID]]    [INOUT] _
      Dati[CLASSE[CLASSE_DATI]]    [INOUT] _
      ProprietaCorrente[STRING]] [IN]      ' Proprietà corrente

Switch ProprietaCorrente

    Case This.Prop1.PropertyId() Prop1()
    Case This.Prop2.PropertyId() Prop2()
    Case This.Prop3.PropertyId() Prop3()

EndSwitch

' Copia dei valori da classe dati a classe video
Dati.Clone(This, 1)

End

'@SRP Prop1()

Dati.Deriva(Dati.Prop1.PropertyId())

Return
```

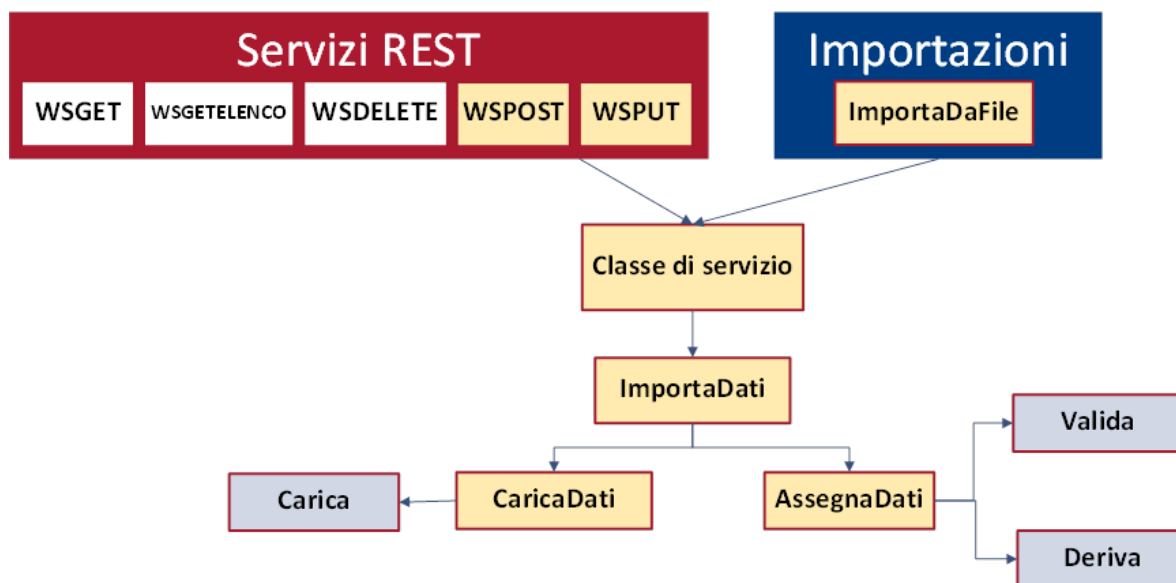
Quale griglia utilizzare e quando

Il seguente schema riassume quale tipologia di griglia utilizzare a seconda delle esigenze:

Casistica	Griglia Standard	Griglia Avanzata	Griglia Avanzata da Query	Griglia di input avanzata
Inputazione dei dati su fonte dati collezione, o comunque fonte dati uniforme trasferibile in una collezione				X
Visualizzazione dati su fonte dati collezione				X
Visualizzazione dati da fonte dati query, senza ciclo di inserimento per modifiche manuali sui dati			X	
Visualizzazione dei dati con fonte dati struttura dinamica o senza fonte dati		X		
Visualizzazione dati da fonte dati query, con ciclo di inserimento manuale per intervenire sui dati estratti		X		
Fonte dati non omogenea(formati diversi per cella nella stessa colonna)	X			

Basandosi su questo specchietto, è necessario prendere come **best practice** che, rispetto a tutte e 4 le griglie, sono sempre da preferire la **griglia di input avanzata** e la **griglia avanzata da query**.

Servizi e importazioni



Classe di servizio

La classe di servizio è la classe che rappresenta come viene esposta l'entità verso l'esterno.

Per esterno si intende sia i servizi web che l'importazione/esportazione da file.

Come definirla

Il nome della classe di servizio deve essere uguale a quello della rispettiva classe dati con il suffisso "_SRV".

La classe conterrà solo i campi che si vogliono esporre verso l'esterno.

Sulla classe vanno creati i vari metodi che serviranno ad importare l'entità e ad esporla tramite servizi.

Questa classe va usata anche nelle firme dei metodi dei servizi REST.

La classe _SRV rappresenta sempre l'intera entità, con tutti i suoi sottolivelli.

Qualora si voglia esporre una classe di servizio per rappresentare l'entità in modo piatto, senza i suoi sottolivelli, va creata una classe con il suffisso _LISTA_SRV.

Su tale classe va previsto soltanto il metodo WSGetElenco con gli stessi standard previsti per la normale classe di servizio.

💡 Utilizza la funzione "Genera classi SRV" presente in AGIS. Ulteriori dettagli sono presenti in [questa scheda](#).

Specializzazioni

Qualora ci siano dei campi che non si vogliono esporre per alcuni verbi del servizio è possibile fare delle apposite classi per tali metodi. Questo permette di fornire una documentazione più precisa agli utilizzatori del servizio. Di contro, però, aumenta la complessità di gestione, manutenzione e personalizzazione, quindi questa soluzione va ponderata in base alla complessità della classe.

Nomenclature standard:

- _SRV_WS<verbo>: Specializzazione da usare per uno specifico verbo del servizio
- _SRV_WS: Specializzazione comune a più verbi del servizio
- _SRV_IMP: Specializzazione da usare per l'importazione

Esempio

Poniamo il caso che si debba implementare un servizio e un'importazione di un'entità che ha un id tecnico e un codice applicativo.

Se per tutti i verbi del servizio si vuole esporre solo l'id tecnico e nell'importazione solo il codice, allora bisogna creare le seguenti classi:

- _SRV con tutti i campi
- _SRV_WS senza il codice applicativo
- _SRV_IMP senza l'id tecnico

Riferimenti

Metodi

Metodi

Metodo	Contesto	Tipo	Descrizione
<u>AssegnaDati</u>	Oggetto	BL	Assegna i valori della classe di servizio sulla classe dati e li valida
<u>CaricaDati</u>	Statico	BL	Controlla l'esistenza dell'entità su DB e la carica, restituendo eventuali errori in base alla modalità di acquisizione
<u>ImportaDati</u>	Oggetto	BL	Gestisce il flusso di importazione/acquisizione dell'entità
<u>WSDelete</u>	Statico	BL	Metodo DELETE del servizio
<u>WSGet</u>	Statico	BL	Metodo GET del servizio
<u>WSGetElenco</u>	Statico	BL	Metodo GET che restituisce una collezione
<u>WSPost</u>	Statico	BL	Metodo POST del servizio
<u>WSPut</u>	Statico	BL	Metodo PUT del servizio

AssegnaDati

Si occupa di assegnare i valori dalla classe di servizio alla classe dati.

Il metodo deve assegnare ogni proprietà singolarmente soltanto se la proprietà è assegnabile (ovvero gestita e non protetta) e se è stata valorizzata dal chiamante.

Il test di valorizzazione del chiamante viene fatto tramite il metodo `GetForcedValue()` combinato in Or con `IsModified()`. Se l'importazione arriva da servizio, è il servizio stesso a impostare i campi passati come "forced". Se invece l'importazione arriva da file, deve essere il chiamante a chiamare il `SetForcedValue()` su tutti i campi vuoti ma presenti nel file e quindi significativi.

Se la proprietà deve essere assegnata, va poi validata prima con un'eventuale validazione tecnica (metodo `ValidateFromClass()`) e poi chiamando il metodo `Valida()` per la validazione applicativa.

Se la validazione non ha dato errori va chiamato `Deriva()` della singola proprietà. Se invece ha dato errori che vanno forzati per l'importazione, essi devono essere convertiti in `Warning` prima di chiamare il `Deriva()`.

Firma

Definizione BC			
'@SBP	This[CLASSE[<classe>_SRV]]	[INOUT]	-
	Dati[CLASSE[<classe>]]	[INOUT]	-
InterrompiAlPrimoErrore[TIPO[BCBOOL]]		[IN]	[VER=1] [DEFAULT=#TRUE]

Parametri

This
Oggetto che della classe di servizio che contiene i valori dell'entità.

Dati
Oggetto della classe dati sul quale assegnare i valori.

InterrompiAlPrimoErrore
Flag che indica se interrompere l'importazione al primo errore riscontrato oppure andare avanti e restituirli tutti.
E' un parametro versionato perché di default deve essere `#True` in modo tale che le acquisizioni da servizio siano più prestanti possibile.
Per le importazioni da file si può scegliere di dare più importanza al restituire tutti gli errori al chiamante.

Esempio

BC

```
'@SBP This[CLASSE[<classe>_SRV]] [INOUT] _
    Dati[CLASSE[<classe>]] [INOUT] _
    InterrompiAlPrimoErrore[TIPO[BCBOOL]] [IN] [VER=1] [DEFAULT=#True]

    AssegnaDati()

End

'@SRP AssegnaDati()

    ' Se l'entità è nuova assegno i campi chiave
    If Dati.IsNew() Then
        AssegnaChiave()
        If Dati.HasError(#ERRORE) Then ExitSr
    EndIf

    AssegnaDatiProp1()
    If Dati.HasError(#ERRORE) And InterrompiAlPrimoErrore Then ExitSr

    AssegnaDatiProp2()
    If Dati.HasError(#ERRORE) And InterrompiAlPrimoErrore Then ExitSr

    AssegnaDatiProp3()
    If Dati.HasError(#ERRORE) And InterrompiAlPrimoErrore Then ExitSr

Return
```

BC

```
'@SRP AssegnaDatiProp1()

    If (This.Prop1.GetForcedValue() = 1 Or This.Prop1.IsModified()) And Dati.Prop1 <> This.Prop1
And IsProprietaAssegnabile(Dati.Prop1.PropertyId()) Then
    ' Assegno i dati dalla classe di servizio e li valido
    Dati.Prop1 = This.Prop1
    Dati.Prop1.ValidateFromClass() '^ Validazione tecnica per obbligatori e valori ammessi
    If Not Dati.Prop1.HasError(#ERRORE) Then Dati.Valida(Dati.Prop1.PropertyId())

    ' Se la proprietà è valida chiamo la derivazione
    If Not Dati.Prop1.HasError(#ERRORE) Then Dati.Deriva(Dati.Prop1.PropertyId())
    EndIf

Return
```

BC

```
'@SRP IsProprietaAssegnabile(idProprieta[STRING] [IN]) AS isAssegnabile[TIPO[BCBOOL]]

    isAssegnabile = #True
    If Not Dati.IsProprietaGestita(idProprieta, errore[CLASSE[BC_VID_ERROR]] [OUT]) Or
Dati.IsProprietaProtetta(idProprieta, errore [OUT]) Then
        Dati.SetError(idProprieta, #INFO, errore.COD, errore.MSG)
        isAssegnabile = #False
    EndIf

Return
```

Riferimenti

Metodo [ImportaDati](#)

Metodo [IsProprietaProtetta](#)

CaricaDati

Metodo che controlla l'esistenza dell'entità su DB e la carica.
In base alla modalità di acquisizione ricevuta può restituire un errore:
Se è stata indicata la modalità di Variazione, l'entità deve essere presente su DB. Se è stata indicata la modalità di Inserimento, l'entità non deve essere presente su DB.

Firma

Definizione BC			
'@SBP	Chiave	[INOUT]	_
	Modalita[TIPO[BCTD_SRV_TIPOACQUISIZIONE]]	[IN]	_
	Errore[CLASSE[BC_VID_ERROR]]	[OUT]	_
	AS Dati[CLASSE[<classe>]]		

Parametri

Chiave
Uno o più parametri che rappresentano i campi chiave dell'entità.

Modalita
Indica la modalità di acquisizione (solo Variazione, solo Inserimento, Variazione o Inserimento).

Errore
Errore riscontrato (entità presente o non presente).

Dati
Entità caricata (vuota se non presente).

Esempio

```
BC

'@SBP Codice [IN] _
      Modalita[TIPO[BCTD_SRV_TIPOACQUISIZIONE]] [IN] _
      Errore[CLASSE[BC_VID_ERROR]] [OUT] _
      AS Dati[CLASSE[]]

' Lettura dell'entità da db
Dati.Carica(Codice, 0, #FALSE, Errore[CLASSE[BC_VID_ERROR] COLLEZIONE])

' L'entità esiste ed è ammesso solo inserire un nuovo codice
If Modalita = BCTD_SRV_TIPOACQUISIZIONE.Inserimento And (Not Dati.IsEmpty()) Then

    ' Imposta l'errore di chiave già esistente
    Errore.COD = 1
    Errore.TIP = 1
    Errore.MSG = "Chiave già esistente"

' La categoria non esiste ed è ammesso solo variare un codice
ElseIf Modalita = BCTD_SRV_TIPOACQUISIZIONE.Variazione And Dati.IsEmpty() Then

    ' Imposta l'errore di chiave inesistente
    Errore.COD = 2
    Errore.TIP = 1
    Errore.MSG = "Chiave inesistente"

' L'entità non esiste
ElseIf Dati.IsEmpty() Then

    ' Inizializza l'oggetto
    Dati.Inizializza()
EndIf

End
```


Riferimenti

Metodo [ImportaDati](#)

ImportaDati

Metodo che si occupa di gestire l'acquisizione dell'entità, partendo dalla classe di servizio.
Riceve in input la modalità di acquisizione (Inserimento, Variazione o Entrambi).
Il metodo richiamerà il metodo [CaricaDati\(\)](#) per controllare l'esistenza dell'entità su DB e la coerenza della modalità di acquisizione.
In caso di esito positivo richiamerà il metodo [AssegnaDati\(\)](#), poi la validazione complessiva dell'entità e infine, se non sono stati riscontrati errori, richiama il [Salva\(\)](#) dell'entità.
Restituisce al chiamante l'entità acquisita e l'elenco di errori, warning e info riscontrati.

Firma

Definizione BC				
'@SBP	This[CLASSE[<classe>_SRV]]	[INOUT]		_
	Modalita[TIPO[BCTD_WS_MODALITAACQUISIZIONE]]	[IN]		_
	Errori[CLASSE[BC_VID_ERROR] COLLEZIONE]	[OUT]		_
	InterrompiAlPrimoErrore[TIPO[BCBOOL]]	[IN]	[VER=1] [DEFAULT=#True]	_
	AS Dati[CLASSE[<classe>]]			

Parametri

This
Oggetto che della classe di servizio che contiene i valori dell'entità.

Modalita
Indica la modalità di acquisizione (solo Variazione, solo Inserimento, Variazione o Inserimento).
Va passato al metodo [CaricaDati\(\)](#).

Errori
Collezione di errori riscontrati nel processo di importazione.

InterrompiAlPrimoErrore
Flag che indica se interrompere l'importazione al primo errore riscontrato oppure andare avanti e restituirli tutti.
E' un parametro versionato perché di default deve essere #True in modo tale che le acquisizioni da servizio siano più prestanti possibile.
Per le importazioni da file si può scegliere di dare più importanza al restituire tutti gli errori al chiamante.

Dati
Entità acquisita.

Esempio

BC

```
'@SBP This[CLASSE[CLS_ARTICOLO_SRV]] [INOUT] _
Modalita[TIPO[BCTD_SRV_TIPOACQUISIZIONE]] [IN] _
Errori[CLASSE[BC_VID_ERROR] COLLEZIONE] [OUT] _
InterrompiAlPrimoErrore[TIPO[BCBOOL]] [IN] [VER=1] [DEFAULT=#True] _
AS Dati[CLASSE[CLS_ARTICOLO]]

ImportaDati()

'@SRP ImportaDati()

' Caricamento dell'entità e controllo errori
Dati = CLS_ARTICOLO_SRV.CaricaDati(This.CodArt , Modalita, Errore[CLASSE[BC_VID_ERROR]] [OUT])

' Se è stato riscontrato un errore lo segnalo ed esco
If Not Errore.IsEmpty() Then
    Errori.AGGIUNGI_ERRORE(Errore [INOUT])
    ExitSr
EndIf

' Valorizza i dati dell'entità con i valori della classe di servizio
This.AssegnaDati(Dati [INOUT], InterrompiAlPrimoErrore)

' Validazione complessiva dell'entità
If Not Dati.HasError(#ERRORE) Or Not InterrompiAlPrimoErrore Then Dati.Valida("")

If Dati.HasError() Then
    ' Recupera gli errori/warning/info rilevati
    Errori.AGGIUNGI_ERRORI(Dati.GetErrors(1) [INOUT])
    ' Se ci sono stati errori bloccanti esco
    If Dati.HasError(#ERRORE) Then ExitSr
EndIf

' Salva l'entità su db
DIM utilizzaTransazione[TIPO[BCBOOL]]
Dati.Salva(utilizzaTransazione [IN])

Return
```

Riferimenti

- Metodo [CaricaDati](#)
- Metodo [AssegnaDati](#)

WSDelete

Metodo DELETE del servizio.

Firma

Definizione BC			
	'@SBP	Chiave[INT]	[IN] _
		Parametri	[IN] _
		Risposta[CLASSE[BC_WS_RISPOSTA]]	[OUT]

Parametri

Chiave
Uno o più parametri che rappresentano i campi chiave dell'entità.

Parametri - *Opzionale*

Uno o più eventuali parametri aggiuntivi in input per il servizio.

Risposta

Oggetto che contiene l'esito del servizio

WSGet

Metodo GET del servizio che restituisce un oggetto singolo.

Firma

Definizione BC			
'@SBP	Chiave[INT]	[IN]	_
	Dati[CLASSE[<classe>_SRV]]	[OUT]	_
	Risposta[CLASSE[BC_WS_RISPOSTA]]	[OUT]	

Parametri

Chiave

Uno o più parametri che rappresentano i campi chiave dell'entità.

Dati

Oggetto che contiene i valori dell'entità da restituire.

Di norma va usata la classe di servizio _SRV.

Qualora ci siano dei campi che non si vogliono esporre è possibile fare una classe apposita avente nome <classe>_WSGET.

Questo permette di fornire una documentazione più precisa agli utilizzatori del servizio. Di contro, però, aumenta la complessità di gestione, manutenzione e personalizzazione, quindi questa soluzione va ponderata in base alla complessità della classe.

-

Risposta

Oggetto che contiene l'esito del servizio

WSGetElenco

Metodo GET del servizio che restituisce una collezione.

Firma

Definizione BC			
'@SBP	Skip[INT]	[IN]	_
	Take[INT]	[IN]	_
	Parametri		_
	Dati[CLASSE[<classe>_SRV] COLLEZIONE]	[OUT]	_
	NumRecord[INT]	[OUT]	_
	Risposta[CLASSE[BC_WS_RISPOSTA]]	[OUT]	

Parametri

Skip

Parametro opzionale da indicare se si intende gestire la paginazione. Indica il numero di elementi da skippare.

Take

Parametro opzionale da indicare se si intende gestire la paginazione. Indica il numero di elementi da leggere.

Parametri

Uno o più parametri di filtro.

Dati

Collezione di oggetti dell'entità da restituire.

Di norma come tipo va usata la classe di servizio _SRV.

Qualora ci siano dei campi che non si vogliono esporre è possibile fare una classe apposita avente nome <classe>_WSGET. Questo permette di fornire una documentazione più precisa agli utilizzatori del servizio. Di contro, però, aumenta la complessità di gestione, manutenzione e personalizzazione, quindi questa soluzione va ponderata in base alla complessità della classe.

NumRecord

Restituisce al chiamante il numero di record estratti dalla query

Risposta

Oggetto che contiene l'esito della chiamata al servizio

WSPost

Metodo POST del servizio.
Il metodo dovrà richiamare il metodo ImportaDati in modalità Inserimento.

Firma

Definizione BC			
'@SBP	Parametri[CLASSE[<classe>_SRV]]	[IN]	_
	Risposta[CLASSE[BC_WS_RISPOSTA]]	[OUT]	_
	AS Key[INT]	[OUT]	

Parametri

Parametri

Oggetto che contiene i valori dell'entità.
Di norma va usata la classe di servizio _SRV.
Qualora ci siano dei campi che non si vogliono esporre è possibile fare una classe apposita avente nome <classe>_WSPOST. Questo permette di fornire una documentazione più precisa agli utilizzatori del servizio. Di contro, però, aumenta la complessità di gestione, manutenzione e personalizzazione, quindi questa soluzione va ponderata in base alla complessità della classe.

Risposta

Oggetto che contiene l'esito del servizio.

[Key]

Valore della chiave dell'entità inserita.
Opzionale, va messo solo se l'entità ha una chiave autoincrementale o comunque definita in automatico in fase di salvataggio.

WSPut

Metodo PUT del servizio.
Il metodo dovrà richiamare il metodo ImportaDati in modalità Variazione.

Firma

Definizione BC			
'@SBP	Parametri[CLASSE[<classe>_SRV]]	[IN]	_
	Risposta[CLASSE[BC_WS_RISPOSTA]]	[OUT]	

Parametri

Parametri

Oggetto che contiene i valori dell'entità.
Di norma va usata la classe di servizio _SRV.
Qualora ci siano dei campi che non si vogliono esporre è possibile fare una classe apposita avente nome <classe>_WSPUT. Questo permette di fornire una documentazione più precisa agli utilizzatori del servizio. Di contro, però, aumenta la complessità di gestione, manutenzione e personalizzazione, quindi questa soluzione va ponderata in base alla complessità

della classe.

Risposta

Oggetto che contiene l'esito del servizio.

Consumo di servizi

Per il consumo di servizi REST, sia di terze parti che interni, va creata una classe per ogni entità.

Questa la nomenclatura prevista:

<Prefisso prodotto>_API_<Prodotto chiamato>_<entità>.

Esempio:

ES_API_SPORTELLO_DITTA

La classe conterrà i metodi corrispondenti ai verbi del servizio da chiamare.

Per i metodi trasversali a più entità, come ad esempio il reperimento dell'url, va creata un'apposita classe statica con questo nome:

<Prefisso prodotto>_API_<Prodotto chiamato>_HELPER

Esempio:

ES_API_SPORTELLO_HELPER

Elaborazioni

Per le elaborazioni deve essere creata un'apposita classe che contiene come proprietà le varie opzioni dell'elaborazione.

Tale classe deve rispettare la nomenclatura: <nome entità>_OPZIONI_<elaborazione>.

Se l'elaborazione prevede anche i filtri dell' entità, è possibile riutilizzare al suo interno la Classe di filtri.

Nella classe dell'elaborazione deve essere creato il metodo Dettaglio (e/o DettaglioWeb) con criteri analoghi al metodo previsto sulle entità.

Il richiamo dell'elaborazione deve avvenire nel metodo Dettaglio alla conferma della videata, ma l'elaborazione vera e propria deve essere collocata in uno specifico metodo di Business logic.

Questo metodo va fatto sull'entità se l'elaborazione è legata ad un entità, altrimenti va previsto sulla classe dell'elaborazione.

Esempio

Stampa di un entità:

La classe di elaborazione si chiamerà <Entità>_OPZIONI_STAMPA e avrà il metodo Dettaglio che al suo interno richiamerà il metodo <Entità>.Stampa passandogli la classe di elaborazione.

Gestione filesystem

- [File protetti](#)

File protetti

I file da proteggere devono essere archiviati in un uno zip protetto con password.

I metodi da utilizzare sono ComprimiContestoProtetto e EstraiContestoProtetto della classe BC_ZIPFILE_HELPER.

Per usare questi metodi è necessario identificare il "contesto applicativo", ovvero una stringa univoca da passare ai due metodi.

Questa stringa va usata per creare una cartella sotto la %TEMP% (vedi BCSystemInfo.GetTempDir) in cui collocare il/i file da proteggere.

La stringa deve essere poi passata ai metodi sopracitati che la usano per ricavare la cartella (concatenando il contesto alla %TEMP%) in cui ci sono i file da comprimere o nella quale vanno estratti.

Va scelta anche un'altra stringa che rappresenta il valore da usare per la creazione della password, in modo che tutti gli zip creati in quel contesto abbiano la stessa password.

La password finale viene composta aggiungendo una costante di ambiente a questa stringa.

Lettura abilitazione IDS

Per la gestione delle abilitazioni IDS va fatta una classe statica di prodotto avente nome:

<sigla prodotto>_IDS_HELPER.

Per ogni singola funzionalità da controllare va fatto un metodo statico avente nome `Is<nomefunzionalità>Abilitata` (ad es: `IsGestioneFattureAbilitata`).

Al suo interno il metodo deve eseguire la '@CHECKIDS.

La firma del metodo deve essere strutturata in questo modo:

Definizione BC
'@SBP DataAbilitazioneScaduta[DATE] [OUT] [VER=1] _
AS Abilitata[TIPO[BCBOOL]]

Il metodo deve restituire `#True` se la funzione è abilitata, anche se è scaduta.

Se è scaduta deve essere valorizzato il parametro `DataAbilitazioneScaduta`, che altrimenti varrà 0.